Team Note of HyeonWooJin

Dohyeon Kim, Chanwoo Cho, Seokjin Lee

Compiled on October 16, 2026

Contents

1 Basic Implementation **1**
1.1 Main Template . . . 1

2 Math **1**
2.1 Arithmetic & Number Theory . . . 1
2.2 Chinese Remainder Theorem . . . 2
2.3 FPS + Bostan-Mori . . . 2
2.4 Bitwise Convolution(FWHT) . . . 3
2.5 Linear Sieve . . . 3
2.6 Miller-Rabin & Pollard-Rho . . . 3
2.7 XOR Basis . . . 3
2.8 Xudyh Sieve . . . 4
2.9 Discrete Log . . . 4
2.10 Discrete Root . . . 4
2.11 Gaussian Elimination . . . 5

3 Data Structure **5**
3.1 Erasable Priority Queue . . . 5
3.2 Segment Tree . . . 5
3.3 1D & 2D Fenwick Tree . . . 6
3.4 Persistent Segment Tree . . . 6
3.5 Link-Cut Tree . . . 6
3.6 Li Chao Tree . . . 8
3.7 DSU (Weighted) . . . 8
3.8 Trie . . . 8
3.9 Disjoint Sparse Table . . . 9
3.10 Mo's Algorithm . . . 9

4 Graph **9**
4.1 Bellman Ford . . . 9
4.2 Eulerian Path . . . 9
4.3 Offline Dynamic Connectivity . . . 10
4.4 Euler Tour / Subtree Flattening . . . 10
4.5 HLD . . . 10
4.6 Virtual Tree . . . 11
4.7 Centroid Decomposition . . . 11
4.8 Tree Isomorphism . . . 11
4.9 Bipartite Matching . . . 11
4.10 Hungarian . . . 12
4.11 Dinic . . . 12
4.12 MCMF . . . 13
4.13 Circulation . . . 13
4.14 SCC . . . 13
4.15 2-sat . . . 13
4.16 BCC . . . 14
4.17 Push Relabel . . . 14
4.18 Gomory Hu Tree . . . 15
4.19 General Matching . . . 15

5 DP **15**
5.1 LIS / 3D LIS . . . 15
5.2 Convex Hull Trick . . . 16
5.3 Linear CHT . . . 16
5.4 D&C optimization . . . 16
5.5 Knuth optimization . . . 16
5.6 Monotone Queue optimization . . . 17
5.7 Aliens Trick . . . 17
5.8 Sum Over Subsets . . . 17
5.9 Berlekamp Massey . . . 17

6 Geometry **18**
6.1 Geometry Template . . . 18
6.2 Closest Pair . . . 18
6.3 Convex Hull . . . 18
6.4 Convex Hull 3d . . . 18
6.5 Rotating Calipers . . . 18
6.6 Point in Polygon . . . 19
6.7 Angular sort . . . 19
6.8 Linear Minkowski Sum . . . 19
6.9 Smallest Enclosing Circle . . . 19
6.10 Geometric Intersections . . . 19
6.11 Half Plane Intersection . . . 20
6.12 Manhattan MST . . . 20

7 String **20**
7.1 Aho-Corasick . . . 20
7.2 Rolling Hash . . . 21
7.3 KMP . . . 21
7.4 Manacher . . . 21
7.5 Suffix Array . . . 21
7.6 Z-algorithm . . . 21
7.7 Eertree . . . 22
7.8 Suffix Automaton . . . 22

8 Policy-Based Data Structures(pbds) **22**
8.1 Hash map (pb_ds) . . . 22
8.2 Ordered Set (pb_ds) . . . 22
8.3 Priority Queue (pb_ds) . . . 22
8.4 Rope . . . 23

9 Misc **23**
9.1 Custom Hash . . . 23
9.2 Fast I/O . . . 23
9.3 Random . . . 23
9.4 Ternary Search . . . 23
9.5 Some tricks . . . 23

10 Checklist + Useful Info **24**
10.1 Highly Composite Numbers, Large Prime . . . 24
10.2 Useful Stuff . . . 24
10.3 자주 쓰이는 문제 접근법 . . . 25
10.4 DP 최적화 접근 . . . 25
10.5 Graph Matching(Graph with $|V| \leq 500$) . . . 25
10.6 Mincut 모델링 . . . 25

1 Basic Implementation

1.1 Main Template

```
// alias gpp='g++ -std=c++20 -O2 -g -Wall -Wshadow=local
-Wno-unused-variable -DLOCAL -fsanitize=address,undefined'
#include <bits/stdc++.h> // #include <bits/extc++.h> for pbds
#pragma GCC optimize ("O3,unroll-loops")
#pragma GCC target ("avx,avx2,fma")
#define fastio cin.tie(0)->sync_with_stdio(0)
#define all(x) (x).begin(),(x).end()
#define rall(x) (x).rbegin(),(x).rend()
#define compress(v) sort(all(v)), v.erase(unique(all(v)),
v.end())
#define sz(x) (int)(x).size()
using namespace std;
using ll = long long;
```

2 Math

2.1 Arithmetic & Number Theory

Usage: Basic Arithmetic, Multiplicative Order, Power Tower, Floor Sum, Binomial Coefficient

```
ll modmul(ll a, ll b, ll m) { return (ll)((__int128)a*b %
m); }
ll modpow(ll b, ll e, ll m) {
  ll ans = 1;
  for (; e; b = modmul(b, b, m), e /= 2)
  if (e & 1) ans = modmul(ans, b, m);
  return ans;
}
ll xgcd(ll a, ll b, ll &x, ll &y) {
  if (!b) return x = 1, y = 0, a;
  ll x1, y1, g = xgcd(b, a % b, x1, y1);
  return x = y1, y = x1 - a / b * y1, g;
}
ll modinv(ll a, ll m) {
  ll x, y;
  ll g = xgcd(a, m, x, y);
  if (g != 1) return -1;
  return (x%m + m) % m;
}
ll phi(ll n) {
  ll res = n;
  for (ll i = 2; i*i <= n; i++) if (n%i == 0) {
    res -= res / i;
    while (n%i == 0) n /= i;
  } if (n > 1) res -= res / n;
  return res;
}
// Multiplicative Order: a^k == 1 (mod m)인 최소 양의 k.
gcd(a,m)=1.
```

```
ll get_order(ll a, ll m) {
  if (gcd(a, m) != 1) return -1;
  ll k = phi(m);
  for (ll p : factor(k))
    if (modpow(a, k/p, m) == 1) k /= p;
  return k;
}
// Power Tower: a[0]^(a[1]^(...)) mod m.
ll power_tower(const vector<ll>& a, int idx, ll m) {
  if (idx == sz(a) || m == 1) return 1;
  auto mul = [](ll x, ll y, ll m) -> ll {
    __int128 v = (__int128)x*y;
    return v < m ? (ll)v : (ll)(v%m)+m;
  };
  auto pow = [&](ll b, ll e) {
    ll ret = 1;
    for (; e; b = mul(b, b, m), e /= 2)
      if (e & 1) ret = mul(ret, b, m);
    return ret;
  };
  return pow(a[idx], power_tower(a, idx+1, phi(m)));
}
cout << power_tower(a, 0, mod) % mod;
// Floor Sum: sum_{i=1}^n floor(n/i). n/i가 같은 구간을 묶음.
ll floor_sum(ll n) {
  ll sum = 0;
  for (ll i = 1, last; i <= n; i = last + 1) {
    last = n / (n / i);
    sum += (n / i) * (last - i + 1);
    // n / x yields the same value for i <= x <= last.
  }
  return sum;
}
// sum_{i=0}^{n-1} floor((a*i+b)/m), O(log m). n,a,b>=0, m>0
ll floor_sum(ll n, ll m, ll a, ll b) {
  ll ret = 0;
  while (true) {
    ret += (__int128)n*(n-1)/2*(a/m) + (__int128)n*(b/m);
    a %= m; b %= m;
    __int128 y = (__int128)a*n + b;
    if (y < m) return ret;
    n = y/m; b = y%m;
    swap(a, m);
  }
}
// nCk: fac[0]=1; finv[N]=modpow(fac[N],M-2,M);
C(n,k)=fac[n]*finv[k]%M*finv[n-k]%M (M prime, N<M, k<0 or k>n
=> 0).
```

2.2 Chinese Remainder Theorem

Usage: 여러 나머지 조건을 동시에 만족하는 정수
Time Complexity: $O(\log N)$

```
// modmul, xgcd
pair<ll,ll> CRT(ll a1, ll m1, ll a2, ll m2) {
  ll x, y, g = xgcd(m1, m2, x, y);
  if ((a2 - a1) % g) return { -1, -1 };
  ll md = m2 / g, k = modmul((a2 - a1) / g, x, md);
  return { a1 + (k < 0 ? k + md : k) * m1, m1 / g * m2 };
}
pair<ll,ll> CRT(const vector<ll>& a, const vector<ll>& m) {
  ll ra = a[0], rm = m[0];
  for (int i = 1; i < (int)m.size(); i++) {
    auto [aa, mm] = CRT(ra, rm, a[i], m[i]);
    if (mm == -1) return { -1, -1 };
    ra = aa; rm = mm;
  }
  return { ra, rm };
}
```

2.3 FPS + Bostan-Mori

Usage: $c_k = \sum_{i+j=k} a_i b_j$ 형태의 합을 한꺼번에 계산
Time Complexity: $O(N \log N)$

```
struct Mint {
  static constexpr ll M = 998244353;
  using V = ll; V val;
  Mint(V y = 0) : val(y % M) { if (val < 0) val += M; }
  operator V() const { return val; }
  Mint operator-() const { return Mint(-val); }
  Mint operator+(auto r) const { return Mint(*this) += r; }
  Mint operator-(auto r) const { return Mint(*this) -= r; }
  Mint operator*(auto r) const { return Mint(*this) *= r; }
  Mint operator/(auto r) const { return Mint(*this) /= r; }
  Mint& operator+=(Mint r) { val += r.val; if (val >= M) val
  -= M; return *this; }
  Mint& operator-=(Mint r) { val -= r.val; if (val < 0) val
  += M; return *this; }
  Mint& operator*=(Mint r) { val = val * r.val % M; return
  *this; }
  Mint& operator/=(Mint r) { return *this *= r.inv(); }
  Mint inv() { assert(val); return inv(val, M); }
  V inv(ll x, ll m) { return x > 1 ? m - inv(m % x, x) * m /
  x : 1; }
  Mint pow(auto y) {
    if (y < 0) return inv().pow(-y);
    Mint r = 1, x = *this;
    for (; y; y /= 2, x *= x) if (y % 2) r *= x;
    return r;
  }
  friend ostream& operator<<(ostream& os, const Mint& x) {
  return os << x.val; }
  friend istream& operator>>(istream& is, Mint& x) { ll v;
  is >> v; x = v; return is; }
};
using Poly = vector<Mint>;
// coef[i]: x^i 계수. resize(N): 앞 N개 계수 유지(부족하면 0).
// inv/log/exp는 현재 size()=N을 기준으로 mod x^N 계산.
struct FPS {
  Poly coef;
  FPS(Poly a) : coef(move(a)) {}
  FPS(int n = 0) : coef{n} {}
  int size() { return coef.size(); }
  int deg() { return size()-1; }
  Mint& operator[](int index) { assert(index < coef.size());
  return coef[index]; }
  FPS operator+(auto f) {
    FPS res = *this; res.resize(max(size(), f.size()));
    for (int i = 0; i < f.size(); i++) res[i] += f[i];
    return res;
  }
  FPS operator-(auto f) {
    FPS res = *this; res.resize(max(size(), f.size()));
    for (int i = 0; i < f.size(); i++) res[i] -= f[i];
    return res;
  }
  // 다항식 곱셈(컨볼루션). 결과 길이 = size()+f.size()-1.
  FPS operator*(auto f) { return FPS(polymul(coef, f.coef)); }
  FPS &operator+=(auto f) { return *this = *this + f; }
  FPS &operator-=(auto f) { return *this = *this - f; }
  FPS &operator*=(auto f) { return *this = *this * f; }
  void resize(int n) { coef.resize(n); }
  // 뒤의 0 제거. 계수 하나는 남김.
  void shrink() { while (size() > 1 && coef.back() == 0)
  coef.pop_back(); }
  // 다항식 거듭제곱. 앞 N개만 필요하면 매 곱셈 후 resize(N)
  FPS power(ll k) {
    FPS res(1), f = *this;
    for (; k; k /= 2, f *= f) if (k % 2) res *= f;
    return res;
  }
  // 1/f mod x^N. 조건: f[0] != 0.
  FPS inv() { assert(coef[0] != 0);
    FPS g(Poly{coef[0].inv()});
    for (int siz = 1; siz < size(); siz *= 2) {
      FPS f = *this; f.resize(siz*2);
      g *= FPS(2) - f*g; g.resize(siz*2);
    }
    g.resize(size()); return g;
  }
  // log(f) mod x^N. 조건: f[0] = 1.
  FPS log() { assert(coef[0] != 0);
    FPS g = differenciate() * inv();
    g.resize(size()-1); return g.integrate();
  }
  // 미분. 결과 길이 N-1.
  FPS differenciate() {
    FPS res; res.resize(size() - 1);
    for (int i = 1; i < size(); i++) res.coef[i - 1] =
    coef[i] * i;
    return res;
  }
  // 적분. 적분상수 0, 결과 길이 N+1.
  FPS integrate() {
    FPS res; res.resize(size() + 1);
    for (int i = 1; i <= size(); i++) res.coef[i] = coef[i -
    1] * Mint(i).inv();
    return res;
  }
  // 형식적 지수 exp(f) mod x^N. 조건: f[0] = 0.
  FPS exp() { assert(coef[0] == 0);
    FPS g(1);
    for (int siz = 1; siz < size(); siz *= 2) {
      FPS f = *this; f.resize(siz*2); g.resize(siz*2);
      g *= f + FPS(1) - g.log(); g.resize(siz*2);
    }
```

```
    g.resize(size()); return g;
  }
  void print() { for (int i = 0; i < size(); i++) cout <<
  coef[i] << ' '; cout << endl; }
  Mint evaluate(Mint x) {
    Mint res = 0;
    for (int i = size()-1; i >= 0; i--) res = res*x + coef[i];
    return res;
  }
  private:
  void ntt(Poly &P, bool inv, Mint g) {
    int n = P.size(); if (n == 1) return;
    for (int i = 1, j = 0; i < n; i++) {
      int bit = n >> 1; for (; j & bit; bit >>= 1) j ^= bit;
      j ^= bit; if (i < j) swap(P[i], P[j]);
    }
    vector<Mint> w(n / 2); w[0] = 1;
    for (int i = 1; i < n / 2; i++) w[i] = w[i - 1] * g;
    for (int i = 1; i < n; i <<= 1) {
      int nd = n / (2 * i);
      for (int j = 0; j < n; j += i << 1) {
        for (int k = 0; k < i; k++) {
          Mint tmp = P[i + j + k] * w[nd * k];
          P[i + j + k] = P[j + k] - tmp; P[j + k] += tmp;
        }
      }
    }
    if (inv) {
      Mint invn = Mint(n).inv();
      for (auto &x : P) x *= invn;
    }
  }
  Poly polymul(Poly a, Poly b) {
    if (a.empty() || b.empty()) return {};
    int n = a.size()+b.size()-1, N = 1;
    while (N < n) N *= 2;
    a.resize(N); b.resize(N);
    Mint g = Mint(3).pow(998244353 / N);
    ntt(a, false, g); ntt(b, false, g);
    for (int i = 0; i < N; i++) a[i] *= b[i];
    ntt(a, true, g.inv()); a.resize(n); return a;
  }
};
// Bostan-Mori: 유리 생성함수 P/Q의 x^k 계수. Q[0]!=0, k>=0.
// d=max(deg P, deg Q), O(d log d log(k+1)). 현재 연산은 mod
998244353.
// 용도: 차수는 작고 인덱스 k는 큰 선형 점화식의 항 계산.
// a[n]=c[1]a[n-1]+...+c[d]a[n-d]일 때:
// A={a[0],...,a[d-1]}, Q={1,-c[1],...,-c[d]}.
// P=FPS(A)*Q; P.resize(d); Bostan_Mori(P,Q,k) = a[k].
Mint Bostan_Mori(FPS P, FPS Q, ll k) {
  while (k) { FPS R = Q;
    for (int i = 1; i < R.size(); i+=2) R[i] = -R[i];
    P *= R; Q *= R; Poly tmp;
    for (int i = k%2; i < P.size(); i+=2) tmp.push_back(P[i]);
    P.coef.swap(tmp); tmp.clear();
    for (int i = 0; i < Q.size(); i+=2) tmp.push_back(Q[i]);
    Q.coef.swap(tmp); k /= 2;
  }
  return P.size() ? P[0] / Q[0] : Mint(0);
}
```

2.4 Bitwise Convolution(FWHT)

Time Complexity: $O(N \log N)$

```
// C[k] = sum_{i op j = k} A[i]*B[j]. op: '&', '|', '^'.
// 값·비트마스크의 결합 결과별 개수/가중치 합을 한꺼번에 계산.
void fwht(vector<ll> &a, char op, bool inv = false) {
  const ll M = 998244353;
  int n = sz(a);
  for (int len = 1; len < n; len <<= 1) for (int i = 0; i <
  n; i += len*2) for (int j = 0; j < len; j++) {
    ll u = a[i+j], v = a[i+j+len];
    if (op == '|') a[i+j+len] = (v + (inv ? M-u : u))%M;
    else if (op == '&') a[i+j] = (u + (inv ? M-v : v))%M;
    else a[i+j] = (u+v)%M, a[i+j+len] = (u-v+M)%M;
  }
  if (inv && op == '^') {
    ll r = modpow(n, M-2, M);
    for (auto& x : a) x = x*r%M;
  }
}
vector<ll> bitconv(vector<ll> a, vector<ll> b, char op) {
  const ll M = 998244353;
  int n = 1;
  while (n < max(sz(a), sz(b))) n <<= 1;
  a.resize(n); b.resize(n);
  fwht(a, op); fwht(b, op);
  for (int i = 0; i < n; i++) a[i] = a[i]*b[i]%M;
  fwht(a, op, true); return a;
}
```

2.5 Linear Sieve

Time Complexity: $O(N)$

```
struct Sieve {
  // sp: 최소 소인수, e: i의 최소 소인수 지수, phi: 오일러 피
  함수(1~i 중 i와 서로소인 개수), mu: 뫼비우스 함수, tau: 약수
  개수, sigma: 약수의 합
  vector<int> sp, e, phi, mu, tau, sigma, tmp, primes;
  Sieve(int n) : sp(n+1), e(n+1), phi(n+1), mu(n+1),
  tau(n+1), sigma(n+1), tmp(n+1) {
    phi[1] = mu[1] = tau[1] = sigma[1] = 1;
    for (int i = 2; i <= n; i++) {
      if (!sp[i]) {
        sp[i]=i; primes.push_back(i);
        e[i]=1; phi[i]=i-1; mu[i]=-1; tau[i]=2;
        sigma[i]=tmp[i]=i+1;
      }
      for (int p : primes) {
        if (i*p > n || p > sp[i]) break;
        int m = i*p; sp[m] = p;
        if (i%p == 0) {
          e[m] = e[i]+1; phi[m] = phi[i]*p; mu[m] = 0;
          tau[m] = tau[i]/(e[i]+1)*(e[m]+1);
          tmp[m] = tmp[i]*p+1; sigma[m] =
          sigma[i]/tmp[i]*tmp[m];
          break;
        }
        e[m] = 1; phi[m] = phi[i]*(p-1); mu[m] = -mu[i];
        tau[m] = tau[i]*2; tmp[m] = p+1; sigma[m] =
        sigma[i]*(p+1);
      }
    }
  }
};
```

2.6 Miller-Rabin & Pollard-Rho

Usage: Primality test and integer factorization.
Time Complexity: $O(\log^3 N)/O(N^{1/4})$

```
// modmul, modpow
bool isPrime(ll n) {
  if (n < 2 || n % 6 % 4 != 1) return (n | 1) == 3;
  ll A[] = {2, 325, 9375, 28178, 450775, 9780504, 1795265022},
    s = __builtin_ctzll(n-1), d = n >> s;
  for (ll a : A) {   // ^ count trailing zeroes
    ll p = modpow(a%n, d, n), i = s;
    while (p != 1 && p != n - 1 && a % n && i--)
      p = modmul(p, p, n);
    if (p != n-1 && i != s) return 0;
  }
  return 1;
}
ll pollard(ll n) {
  auto f = [n](ll x) { return modmul(x, x, n) + 3; };
  ll x = 0, y = 0, t = 30, prd = 2, i = 1, q;
  while (t++ % 40 || __gcd(prd, n) == 1) {
    if (x == y) x = ++i, y = f(x);
    if ((q = modmul(prd, max(x,y) - min(x,y), n))) prd = q;
    x = f(x), y = f(f(y));
  }
  return __gcd(prd, n);
}
vector<ll> factor(ll n) {
  if (n == 1) return {};
  if (isPrime(n)) return {n};
  ll x = pollard(n);
  auto l = factor(x), r = factor(n / x);
  l.insert(l.end(), r.begin(), r.end());
  return l;
}
vector<ll> res = factor(N); // factor of N
```

2.7 XOR Basis

Usage: 부분집합 XOR의 생성 가능 여부·최댓값·k번째 값. 원소 하나와의 XOR는 XorTrie.

```
/* using Mask = bitset<MAX_N>; */
template <typename T = ll, int B = 62> struct Basis {
  T b[B]{}; int sz = 0;
  /* Mask mask[B]{}; */
  // 기저 원소 삽입, O(B)
  bool insert(T x /*, int id*/) {
    /* Mask cur; cur.set(id); */
    for (int i = B-1; i >= 0; i--) {
      if (!(x >> i & 1)) continue;
```

```
    if (!b[i]) {
      b[i] = x; sz++;
      /* mask[i] = cur; */
      return true;
    }
    x ^= b[i];
    /* cur ^= mask[i]; */
  }
  return false;
}
// 만들 수 있는 최댓값, O(B)
T max_xor(T init = 0 /*, Mask &res*/) const {
  /* res.reset(); */
  for (int i = B-1; i >= 0; i--) {
    if ((init ^ b[i]) > init) {
      init ^= b[i];
      /* res ^= mask[i]; */
    }
  }
  return init;
}
// 0 제외 최솟값, O(B)
T min_xor() const {
  for (int i = 0; i < B; i++) if (b[i]) return b[i];
  return 0;
}
// 기저로 줄인 잔여값. res: 사용한 원소들의 마스크.
T reduce(T x /*, Mask* res = nullptr*/) const {
  /* if (res) res->reset(); */
  for (int i = B-1; i >= 0; i--) {
    if (x >> i & 1) {
      x ^= b[i];
      /* if (res) *res ^= mask[i]; */
    }
  }
  return x;
}
// 생성 가능 여부
bool can(T x /*, Mask* res = nullptr*/) const {
  return reduce(x /*, res*/) == 0;
}
// 기약 사다리꼴(RREF) 기저 목록, O(B^2)
vector<T> rref() const {
  T tmp[B]; copy(begin(b), end(b), tmp);
  for (int i = 0; i < B; i++) {
    if (!tmp[i]) continue;
    for (int j = i + 1; j < B; j++) {
      if (tmp[j] >> i & 1) tmp[j] ^= tmp[i];
    }
  }
  vector<T> rb;
  for (int i = 0; i < B; i++) if (tmp[i])
  rb.push_back(tmp[i]);
  return rb;
}
// k번째 최소 XOR 합 (1-idx, 0 포함), O(B^2)
T kth(ll k) const {
  if (k > (1ULL << sz)) return -1;
  auto rb = rref();
  T res = 0;
  for (int i = 0; i < sz; i++) {
    if ((k-1) >> i & 1) res ^= rb[i];
  }
  return res;
}
// x의 순위 (1-idx, 0 포함, 불가능 시 -1), O(B^2)
ll rank(T x) const {
  auto rb = rref();
  ll res = 0;
  for (int i = 0; i < sz; i++) {
    if (x >= (x ^ rb[i])) {
      x ^= rb[i];
      res |= (1LL << i);
    }
  }
  return x ? -1 : res + 1;
}
// 기저 병합, O(B^2)
void merge(const Basis& o) {
  for (int i = 0; i < B; i++) if (o.b[i]) insert(o.b[i]);
}
};
```

2.8 Xudyh Sieve

Usage: 큰 N 까지의 ϕ, μ 누적합
Time Complexity: $O(N^{3/4})$

```
// f의 누적합: g와 f*g의 누적합을 쉽게 계산할 수 있을 때.
// pf(n): 전처리한 f의 누적합, pg(n): g의 누적합, pfg(n):
f*g의 누적합.
// mu 누적합: pg(n)=n, pfg(n)=1.
// phi 누적합: pg(n)=n, pfg(n)=n(n+1)/2.
// th까지 pf 전처리 후 xudyh_sieve(th,pf,pg,pfg).query(N).
template<class T, class F1, class F2, class F3>
struct xudyh_sieve{
  T th; // threshold, 2(single query) ~ 5 * MAXN^2/3
  F1 pf; F2 pg; F3 pfg;
  // prefix sum of f(up to th), g(easy to calc), f*g(easy to
  calc)
  unordered_map<T, T> mp; // f * g means dirichlet conv.
  xudyh_sieve(T th, F1 pf, F2 pg, F3 pfg) : th(th), pf(pf),p
  g(pg), pfg(pfg) {}
  // Calculate the preix sum of a multiplicative f up to n
  T query(T n){ // O(n^2/3)
    if(n <= th) return pf(n); if(mp.count(n)) return mp[n];
    T res = pfg(n);
    for(T low = 2, high = 2; low <= n; low = high + 1){
      high = n / (n / low);
      res -= (pg(high) - pg(low - 1)) * query(n / low);
      res = (res % MOD + MOD) % MOD; // delete
    }
    return mp[n] = res * modpow(pg(1), MOD-2, MOD) % MOD; //
    res / pg(1);
  }
};
```

2.9 Discrete Log

Time Complexity: $O(\sqrt{m} \log m)$

```
// a^x == b (mod m)의 최소 x>=0. 없으면 -1
// m>=1, 합성수·gcd(a,m)>1 지원.
ll discrete_log(ll a, ll b, int m) {
  a = (a%m+m)%m; b = (b%m+m)%m;
  ll k = 1; int cnt = 0;
  for (ll g; (g = gcd(a, (ll)m)) > 1; cnt++) {
    if (b == k) return cnt;
    if (b%g) return -1;
    b /= g; m /= g; k = k*(a/g)%m;
  }
  if (m == 1 || b == k) return cnt;
  int n = (int)sqrt(m) + 1;
  ll step = modpow(a, n, m), cur = b;
  map<ll,int> dec;
  for (int j = 0; j < n; j++, cur = cur*a%m) dec[cur] = j;
  cur = k;
  for (int i = 1; i <= n; i++) {
    cur = cur*step%m;
    auto it = dec.find(cur);
    if (it != dec.end()) return 1LL*i*n - it->second + cnt;
  }
  return -1;
}
```

2.10 Discrete Root

Time Complexity: $O(\sqrt{p} \log p)$

```
// p is prime
int primitive_root(int p) {
  if (p == 2) return 1;
  vector<int> fact;
  int n = p-1;
  for (ll i = 2; i*i <= n; i++) if (n%i == 0) {
    fact.push_back(i);
    while (n%i == 0) n /= i;
  }
  if (n > 1) fact.push_back(n);
  for (int g = 2; g < p; g++) {
    bool chk = true;
    for (int q : fact) if (modpow(g, (p-1)/q, p) == 1) {
      chk = false; break;
    }
    if (chk) return g;
  }
  return -1;
}
// 소수 p에서 x^k == a (mod p)의 해 하나. 없으면 -1.
// k >= 1. 최소 해를 보장하지 않음.
int discrete_root(ll k, ll a, int p) {
  assert(p >= 2 && k >= 1);
  a = (a%p + p) % p;
  if (a == 0) return 0;
  if (p == 2) return 1;
  int phi = p-1, g = primitive_root(p);
  k %= phi;
  int d = phi / gcd(k, (ll)phi);
  if (modpow(a, d, p) != 1) return -1;
  // x = g^y. (g^k)^y == a를 풀며, y는 modulo d.
```

```
  int sq = (int)sqrt(d) + 1;
  ll h = modpow(g, k, p), step = modpow(h, sq, p), cur = 1;
  vector<pair<int,int>> dec(sq);
  for (int i = 1; i <= sq; i++) {
    cur = cur * step % p;
    dec[i-1] = { (int)cur, i };
  }
  sort(all(dec)); cur = a;
  for (int i = 0; i < sq; i++) {
    auto it = lower_bound(all(dec), make_pair((int)cur, 0));
    if (it != dec.end() && it->first == cur) {
      ll y = (1LL*it->second*sq - i) % d;
      return modpow(g, y, p);
    }
    cur = cur * h % p;
  }
  return -1;
}
```

2.11 Gaussian Elimination

```
// modinv
struct Gauss {
  using Mat = vector<vector<ll>>;
  const ll M = 1000000007;
  // 앞 cols개 열을 소거. 반환: {rank, 정사각 계수행렬의 det}.
  // full=true: RREF, false: 전진 소거. 계수는 [0,M).
  pair<int,ll> elim(Mat& a, int cols, bool full = true) {
    int n = sz(a), m = n ? sz(a[0]) : 0, r = 0;
    ll det = 1;
    for (int c = 0; c < cols && r < n; c++) {
      int p = r;
      while (p < n && a[p][c] == 0) p++;
      if (p == n) continue;
      if (p != r) { swap(a[p], a[r]); det = (M-det)%M; }
      det = det*a[r][c]%M;
      ll v = modinv(a[r][c], M);
      for (int j = c; j < m; j++) a[r][j] = a[r][j]*v%M;
      for (int i = full ? 0 : r+1; i < n; i++) {
        if (i == r || a[i][c] == 0) continue;
        ll f = a[i][c];
        for (int j = c; j < m; j++)
          a[i][j] = (a[i][j]-f*a[r][j]%M+M)%M;
      }
      r++;
    }
    return {r, r == cols ? det : 0};
  }
  // a: n*n, b: 우변 행렬. false면 a가 특이행렬.
  bool rref(Mat& a, Mat& b, int n) {
    for (int i = 0; i < n; i++)
      a[i].insert(a[i].end(), all(b[i]));
    bool ok = elim(a, n).first == n;
    for (int i = 0; i < n; i++) {
      b[i].assign(a[i].begin()+n, a[i].end());
      a[i].resize(n);
    }
    return ok;
  }
  // n*(n+1) 확대행렬. 유일해가 없으면 {}.
  vector<ll> solve(Mat a) {
    int n = sz(a);
    if (elim(a, n).first != n) return {};
    vector<ll> x(n);
    for (int i = 0; i < n; i++) x[i] = a[i][n];
    return x;
  }
  // 정사각행렬의 역행렬. 없으면 {}.
  Mat inverse(Mat a) {
    int n = sz(a);
    for (int i = 0; i < n; i++) {
      a[i].resize(2*n); a[i][n+i] = 1;
    }
    if (elim(a, n).first != n) return {};
    for (auto& row : a) row.erase(row.begin(), row.begin()+n);
    return a;
  }
  ll det(Mat a) { return elim(a, sz(a), false).second; }
  int rank(Mat a) {
    return elim(a, a.empty() ? 0 : sz(a[0]), false).first;
  }
};
```

3 Data Structure

3.1 Erasable Priority Queue

```
template <typename T = int, typename Compare = less<T>>
struct EraseablePQ {
  priority_queue<T,vector<T>,Compare> q, del;
  void flush() {
    while (!del.empty() && !q.empty() && q.top() ==
    del.top()) {
      q.pop(); del.pop(); }
  }
  void push(const T& x) { q.push(x); }
  void erase(const T& x) { del.push(x); }
  void pop() { flush(); if (!q.empty()) q.pop(); }
  const T& top() { flush(); return q.top(); }
  int size() const { return int(q.size() - del.size()); }
  bool empty() { return q.size() == del.size(); }
};
```

3.2 Segment Tree

```
// add/set: 구간 덧셈/대입, query: {sum,mn,mx}.
// find/find_kth는 실제 사용하는 인덱스 범위를 지정.
template<typename T = ll> struct SegTree {
  struct Nd { T sum, mn, mx; };
  struct Lz { T add, set; bool has; };
  int n; vector<Nd> tr; vector<Lz> lz;
  SegTree(int n) : n(n), tr(4*n+5), lz(4*n+5, { 0, 0, 0 }) {}
  Nd id() { return { 0, numeric_limits<T>::max(),
  numeric_limits<T>::lowest() }; }
  Nd merge(Nd a, Nd b) { return { a.sum+b.sum,
  min(a.mn,b.mn), max(a.mx,b.mx) }; }
  Lz comp(Lz f, Lz g) { // f(g(x))
    if (f.has) return f;
    g.add += f.add; return g;
  }
  void apply(int nd, int s, int e, Lz f) {
    if (f.has) tr[nd] = { f.set*(e-s+1), f.set, f.set };
    tr[nd].sum += f.add*(e-s+1);
    tr[nd].mn += f.add; tr[nd].mx += f.add;
    lz[nd] = comp(f, lz[nd]);
  }
  void push(int nd, int s, int e) {
    if (!lz[nd].has && lz[nd].add == 0) return;
    if (s != e) {
      int m = (s+e)>>1;
      apply(nd<<1, s, m, lz[nd]);
      apply(nd<<1|1, m+1, e, lz[nd]);
    }
    lz[nd] = { 0, 0, 0 };
  }
  void build(int nd, int s, int e, const vector<T>& a) {
    lz[nd] = { 0, 0, 0 };
    if (s == e) {
      T v = s < sz(a) ? a[s] : 0;
      return void(tr[nd] = { v, v, v });
    }
    int m = (s+e)>>1;
    build(nd<<1, s, m, a); build(nd<<1|1, m+1, e, a);
    tr[nd] = merge(tr[nd<<1], tr[nd<<1|1]);
  }
  void upd(int nd, int s, int e, int l, int r, Lz f) {
    if (l > r || r < s || e < l) return;
    if (l <= s && e <= r) return apply(nd, s, e, f);
    push(nd, s, e); int m = (s+e)>>1;
    upd(nd<<1, s, m, l, r, f); upd(nd<<1|1, m+1, e, l, r, f);
    tr[nd] = merge(tr[nd<<1], tr[nd<<1|1]);
  }
  Nd qry(int nd, int s, int e, int l, int r) {
    if (l > r || r < s || e < l) return id();
    if (l <= s && e <= r) return tr[nd];
    push(nd, s, e); int m = (s+e)>>1;
    return merge(qry(nd<<1, s, m, l, r),
                 qry(nd<<1|1, m+1, e, l, r));
  }
  int walk(int nd, int s, int e, int l, int r, Nd& pref,
  auto& chk) {
    if (l > r || r < s || e < l) return -1;
    if (l <= s && e <= r && !chk(pref, tr[nd])) {
      pref = merge(pref, tr[nd]); return -1;
    }
    if (s == e) return s;
    push(nd, s, e); int m = (s+e)>>1;
    int ret = walk(nd<<1, s, m, l, r, pref, chk);
    return ret != -1 ? ret : walk(nd<<1|1, m+1, e, l, r,
    pref, chk);
  }
  void build(const vector<T>& a) { build(1, 0, n, a); }
  void add(int l, int r, T x) { upd(1, 0, n, l, r, { x, 0, 0
  }); }
  void set(int l, int r, T x) { upd(1, 0, n, l, r, { 0, x, 1
  }); }
  Nd query(int l, int r) { return qry(1, 0, n, l, r); }
  T qsum(int l, int r) { return query(l, r).sum; }
```

```
  T qmin(int l, int r) { return query(l, r).mn; }
  T qmax(int l, int r) { return query(l, r).mx; }
  // [l,i]가 조건을 처음 만족하는 i. 없으면 -1. O(log n).
  // chk(pref,node): 합친 구간의 판정. 확장 시 false -> true만 가능.
  int find(auto chk, int l = 0, int r = -1) {
    if (r == -1) r = n;
    Nd pref = id(); return walk(1, 0, n, l, r, pref, chk);
  }
  // [l,i] 합 >= k인 첫 i. 원소 비음수, k>=1. 없으면 -1.
  int find_kth(T k, int l = 0, int r = -1) {
    if (k <= 0) return -1;
    return find([&](Nd a, Nd b) { return a.sum+b.sum >= k;
    }, l, r);
  }
};
SegTree<ll> seg(sz(a)-1); seg.build(a); // 0-based
seg.add(1, 3, 2); seg.set(2, 4, 3); // a[1..3] += 2, a[2..4] = 3
seg.qsum(1, 3); seg.qmin(1, 3); seg.qmax(1, 3); //합, 최솟값, 최댓값
seg.find_kth(5); // 누적합 >= 5인 첫 idx. 원소 비음수
seg.find([&](auto p, auto v) { return max(p.mx, v.mx) >= 4;
}); // 첫 a[i]>=4
```

3.3 1D & 2D Fenwick Tree

Time Complexity: 1D: $O(\log N)$, 2D: $O(\log N \log M)$

```
struct Fenwick {
  const ll MAXN = 100000;
  vector<ll> tree; int SZ;
  Fenwick(ll n) : tree(n), SZ(n) {}
  Fenwick() : Fenwick(MAXN) {}
  ll query(ll p) { // sum from index 0 to p, inclusive
    ll ret = 0;
    for (; p >= 0; p = (p & (p+1)) - 1) ret += tree[p];
    return ret;
  }
  void add(ll p, ll val) {
    for (; p < SZ; p |= p+1) tree[p] += val;
  }
};
// Ex. inversion counting
vector<ll> v = a; compress(v);
Fenwick bit(sz(v)); ll inv = 0;
for (int i = 0; i < sz(a); i++) {
  int x = lower_bound(all(v), a[i]) - v.begin();
  inv += i - bit.query(x);
  bit.add(x, 1);
}
// Fenwick_2d<int> T(n+1,m+1) for nxm grid indexed from 1
template<class T> struct Fenwick_2d {
  vector<vector<T>> x;
  Fenwick_2d(int n, int m) : x(n, vector<T>(m)) { }
  void add(int k1, int k2, int a) { // x[k] += a
    for (; k1 < sz(x); k1 |= k1+1)
      for (int k = k2; k < sz(x[k1]); k |= k+1) x[k1][k] += a;
  }
  T sum(int k1, int k2) { // return x[0]+...+x[k]
    T s = 0;
    for (; k1 >= 0; k1 = (k1 & (k1 + 1)) - 1)
      for (int k = k2; k >= 0; k = (k&(k+1))-1) s += x[k1][k];
    return s;
  }
};
```

3.4 Persistent Segment Tree

Usage: 정적 배열의 구간 k번째 값·값 범위별 개수·이전/다음 값
Time Complexity: $O(\log N)$ per query

```
// 정적 배열의 구간 값 쿼리용 PST.
// 배열 0-based, [l,r] 양끝 포함, kth의 k는 1-based.
// root[i]: 앞의 i개 원소의 빈도표. [l,r] = root[r+1]-root[l].
// 구축/공간 O(N log M), 쿼리 O(log M). M은 서로 다른 값 수.
struct PST {
  struct Node { int l = 0, r = 0, cnt = 0; };
  vector<Node> tr{{}}; // 0번: 빈 노드
  vector<int> root;
  vector<ll> v;
  int n, m;
  PST(const vector<ll>& a) : root(sz(a)+1), v(a), n(sz(a)) {
    compress(v); m = sz(v);
    for (int i = 0; i < n; i++) {
      int x = lower_bound(all(v), a[i]) - v.begin();
      root[i+1] = add(root[i], 0, m-1, x);
    }
  }
  int add(int p, int s, int e, int x) {
    int u = sz(tr); tr.push_back(tr[p]); tr[u].cnt++;
    if (s == e) return u;
    int mid = (s + e) >> 1;
    if (x <= mid) tr[u].l = add(tr[p].l, s, mid, x);
    else tr[u].r = add(tr[p].r, mid+1, e, x);
    return u;
  }
  int query(int p, int q, int s, int e, int l, int r) const {
    if (l > r || r < s || e < l || p == q) return 0;
    if (l <= s && e <= r) return tr[q].cnt - tr[p].cnt;
    int mid = (s + e) >> 1;
    return query(tr[p].l, tr[q].l, s, mid, l, r)
         + query(tr[p].r, tr[q].r, mid+1, e, l, r);
  }
  int count_idx(int l, int r, int L, int R) const { // 압축값 [L,R] 개수
    assert(0 <= l && l <= r && r < n);
    return query(root[l], root[r+1], 0, m-1, L, R);
  }
  int count_less(int l, int r, ll x) const { // 원소 < x
    int R = lower_bound(all(v), x) - v.begin() - 1;
    return count_idx(l, r, 0, R);
  }
  int count_le(int l, int r, ll x) const { // 원소 <= x
    int R = upper_bound(all(v), x) - v.begin() - 1;
    return count_idx(l, r, 0, R);
  }
  int count(int l, int r, ll lo, ll hi) const { // lo <= 원소 <= hi
    int L = lower_bound(all(v), lo) - v.begin();
    int R = upper_bound(all(v), hi) - v.begin() - 1;
    return count_idx(l, r, L, R);
  }
  ll kth(int l, int r, int k) const { // 중복 포함 k번째 작은 값
    assert(0 <= l && l <= r && r < n && 1 <= k && k <= r-l+1);
    int p = root[l], q = root[r+1], s = 0, e = m-1;
    while (s < e) {
      int mid = (s + e) >> 1, cnt = tr[tr[q].l].cnt -
      tr[tr[p].l].cnt;
      if (k <= cnt) p = tr[p].l, q = tr[q].l, e = mid;
      else k -= cnt, p = tr[p].r, q = tr[q].r, s = mid+1;
    }
    return v[s];
  }
  optional<ll> prev(int l, int r, ll x) const { // x 이하 최댓값
    int k = count_le(l, r, x);
    if (!k) return nullopt;
    return kth(l, r, k);
  }
  optional<ll> next(int l, int r, ll x) const { // x 이상 최솟값
    int k = count_less(l, r, x) + 1;
    if (k > r-l+1) return nullopt;
    return kth(l, r, k);
  }
};
// (r-l+1)-pst.count_less(l,r,x);        // x 이상 개수
// (r-l+1)-pst.count_le(l,r,x);          // x 초과 개수
// pst.kth(l,r,r-l+2-k);                 // k번째 큰 값
// auto x = pst.next(l,r,tar); if(x) cout << *x;
```

3.5 Link-Cut Tree

Usage: 간선 연결·삭제가 있는 숲의 경로 쿼리·갱신. 고정 트리는 HLD
Time Complexity: Amortized $O(\log N)$

```
// MAXN은 (초기 크기 + insert 총 횟수) 이상
const ll INF = 1e18, MAXN = 3e5+5;
struct BBST {
  struct Node {
    int l = 0, r = 0, p = 0, id = -1;
    ll cnt = 0, val = 0, sum = 0, mx = -INF, mn = INF, lz = 0;
    bool flip = false;
  } tr[MAXN];
  int rt = 0, ptr = 0, pos[MAXN];
  void clear() {
    fill(tr, tr+ptr+1, Node{}); rt = ptr = 0;
  }
  int new_nd(ll v = 0, int id = -1) {
    auto &n = tr[++ptr]; n.id = id;
    n.val = n.mx = n.mn = n.sum = v; n.cnt = 1;
    n.l = n.r = n.p = n.lz = 0; n.flip = false;
    if (id != -1) pos[id] = ptr;
    return ptr;
  }
  bool is_rt(int x) { return !tr[x].p || (tr[tr[x].p].l != x
  && tr[tr[x].p].r != x); }
  void update(int x) {
```

```
  if (!x) return;
  auto &n = tr[x], &l = tr[n.l], &r = tr[n.r];
  n.cnt = 1+l.cnt+r.cnt; n.sum = n.val + l.sum + r.sum;
  n.mx = max({n.val, n.l ? l.mx : -INF, n.r ? r.mx : -INF});
  n.mn = min({n.val, n.l ? l.mn : INF, n.r ? r.mn : INF});
}
void apply(int x, ll v) {
  if (!x) return;
  auto &n = tr[x]; n.val += v; n.lz += v;
  n.mx += v; n.mn += v; n.sum += v*n.cnt;
}
void push(int x) {
  if (!x) return;
  auto &n = tr[x];
  if (n.lz) {
    apply(n.l, n.lz); apply(n.r, n.lz); n.lz = 0;
  } if (n.flip) {
    swap(n.l, n.r);
    if (n.l) tr[n.l].flip ^= 1;
    if (n.r) tr[n.r].flip ^= 1;
    n.flip = false;
  }
}
void push_all(int x) {
  if (!is_rt(x)) push_all(tr[x].p);
  push(x);
}
void rotate(int x) {
  int p = tr[x].p, g = tr[p].p, b;
  if (x == tr[p].l) tr[p].l = b = tr[x].r, tr[x].r = p;
  else tr[p].r = b = tr[x].l, tr[x].l = p;
  if (!is_rt(p)) (tr[g].l == p ? tr[g].l : tr[g].r) = x;
  else rt = x;
  tr[x].p = g; tr[p].p = x;
  if (b) tr[b].p = p;
  update(p); update(x);
}
void splay(int x, int tgt = 0) {
  push_all(x);
  while (tr[x].p != tgt && !is_rt(x)) {
    int p = tr[x].p, g = tr[p].p;
    if (g != tgt && !is_rt(p)) {
      if ((tr[p].l == x) == (tr[g].l == p)) rotate(p);
      else rotate(x);
    } rotate(x);
  }
  if (!tgt && is_rt(x)) rt = x;
}
// k번째 원소를 tg(기본 0=루트) 아래로 splay
void find_kth(int k, int tg = 0) {
  int cur = rt;
  while (push(cur), 1) {
    int lc = tr[tr[cur].l].cnt;
    if (k == lc+1) break;
    if (k <= lc) cur = tr[cur].l;
    else k -= lc+1, cur = tr[cur].r;
  }
  splay(cur, tg);
}
```

```
// 구간을 서브트리로 모음 (루트 반환, 1 <= l <= r <= N)
int gather(int l, int r) {
  find_kth(r+2); find_kth(l, rt);
  return tr[tr[rt].l].r;
}
int build(int l, int r, int p, const vector<ll> &v) {
  if (l > r) return 0;
  int mid = (l+r)/2, id = -1; ll val = 0;
  if (mid > 1 && mid < sz(v)+2) val = v[mid-2], id = mid-1;
  int x = new_nd(val, id);
  tr[x].p = p;
  tr[x].l = build(l, mid-1, x, v);
  tr[x].r = build(mid+1, r, x, v);
  update(x); return x;
}
void init(const vector<ll> &v) { // v는 0-idx vector
  clear();
  rt = build(1, sz(v)+2, 0, v);
}
void add_val(int l, int r, ll v) { apply(gather(l, r), v); }
int get_idx(int id) { // ID 원소의 인덱스 반환 (1-idx)
  push_all(pos[id]); splay(pos[id]);
  return tr[tr[rt].l].cnt;
}
ll get_val(int k) { find_kth(k+1); return tr[rt].val; }
void reverse(int l, int r) { tr[gather(l, r)].flip ^= 1; }
// k번째 뒤(0: 앞 / N: 뒤)에 값 v (ID: id) 삽입
void insert(int k, ll v, int id = -1) {
  find_kth(k+1); find_kth(k+2, rt);
  int x = new_nd(v, id), p = tr[rt].r;
  tr[p].l = x; tr[x].p = p;
  update(p); update(rt); splay(x);
}
void erase(int l, int r) {
  int p = tr[gather(l, r)].p;
  tr[p].r = 0; update(p); splay(p);
}
void shift(int l, int r, int k) { // k칸 우측 시프트
  int L = r-l+1; k = (k%L+L) % L;
  if (!k) return;
  reverse(l, r); reverse(l, l+k-1); reverse(l+k, r);
}
int build_val(int l, int r, int p, const
vector<pair<ll,int>> &v) {
  if (l > r) return 0;
  int mid = (l + r) / 2;
  ll val = 0; int id = -1;
  if (mid == 1) val = -INF;
  else if (mid == sz(v)+2) val = INF;
  else val = v[mid-2].first, id = v[mid-2].second;
  int x = new_nd(val, id); tr[x].p = p;
  tr[x].l = build_val(l, mid - 1, x, v);
  tr[x].r = build_val(mid + 1, r, x, v);
  update(x); return x;
}
// (val, id) 사전순 정렬된 v(0-idx)로 트리 생성
void init_val(const vector<pair<ll,int>> &v = {}) {
  clear(); rt = build_val(1, sz(v)+2, 0, v);
}
```

```
// (val, id) >= (s, id) 인 첫 노드를 tg 아래로 splay
int lower_bound(ll s, int id = -1, int tg = 0) {
  int cur = rt, ret = 0;
  while (cur) { push(cur);
    if (tie(tr[cur].val, tr[cur].id) >= tie(s, id)) {
      ret = cur; cur = tr[cur].l;
    }
    else cur = tr[cur].r;
  }
  if (ret) splay(ret, tg);
  return ret;
}
// (val, id) < (s, id) 인 마지막 노드를 tg 아래로 splay
int prev_bound(ll s, int id = -1, int tg = 0) {
  int cur = rt, ret = 0;
  while (cur) { push(cur);
    if (tie(tr[cur].val, tr[cur].id) < tie(s, id)) {
      ret = cur; cur = tr[cur].r;
    }
    else cur = tr[cur].l;
  }
  if (ret) splay(ret, tg);
  return ret;
}
// 값 [ (s1,id1), (s2,id2) ) 구간을 모음 (루트 반환)
int gather_val(ll s1, ll s2, int id1 = -1, int id2 = -1) {
  int r = lower_bound(s2, id2), l = prev_bound(s1, id1, r);
  return tr[l].r;
}
// (v, id)값 노드 정렬 위치에 삽입
void insert_val(ll v, int id = -1) {
  int r = lower_bound(v, id), l = prev_bound(v, id, r);
  int x = new_nd(v, id); tr[l].r = x; tr[x].p = l;
  update(l); update(r); splay(x);
}
void erase_val(ll v, int id = -1) { // (v, id)값 노드 제거
  int r = lower_bound(v, id+1), l = prev_bound(v, id, r);
  tr[l].r = 0; update(l); update(r); splay(l);
}
// === Link-Cut tree ===
// 루트~x 경로를 단일 Splay로 연결
void access(int x) {
  for (int y = 0; x; y = x, x = tr[x].p) {
    splay(x); tr[x].r = y; update(x);
  }
}
void make_rt(int x) { access(x); splay(x); tr[x].flip ^=
1; }
int find_rt(int x) {
  access(x); splay(x);
  while (push(x), tr[x].l) x = tr[x].l;
  splay(x); return x;
}
bool conn(int u, int v) { return find_rt(u) == find_rt(v); }
// link/cur: 간선 (u, v) 추가/제거 (성공 시 true)
bool link(int u, int v) {
  if (conn(u, v)) return false;
  make_rt(u); tr[u].p = v; return true;
```

```
  }
  bool cut(int u, int v) {
    make_rt(u);
    if (find_rt(v) == u && tr[v].p == u && !tr[v].l) {
      tr[v].p = tr[u].r = 0;
      update(u); return true;
    }
    return false;
  }
  void cut(int x) { // x와 부모 노드 간선 제거
    access(x); splay(x);
    if (tr[x].l) {
      tr[tr[x].l].p = 0; tr[x].l = 0; update(x);
    }
  }
  int lca(int u, int v) {
    if (!conn(u, v)) return 0;
    access(u); access(v); splay(u);
    return tr[u].p ? tr[u].p : u;
  }
  // path_xxx: u, v가 연결되어 있을 때만 호출 가능
  Node path_qry(int u, int v) {
    make_rt(u); access(v); splay(v);
    return tr[v];
  }
  void path_upd(int u, int v, ll val) {
    make_rt(u); access(v); splay(v);
    apply(v, val);
  }
};
```

3.6 Li Chao Tree

Usage: 순서 제약 없는 직선 추가·최솟값 쿼리

```
struct LiChaoTree {
  // min LiChao, 정수, 사용하기 전에 init(min_x, max_x) 호출
  // update(0, {a, b}) : 직선 y=ax+b 추가
  // update(0, {a, b}, l, r) : y=ax+b를 x in [l, r]에 추가
  // query(0, k) : x=k에서 최솟값 리턴, 직선 없으면 INF
  // 쿼리, 직선 update : O(log n), 선분 update : O(log^2n)
  struct Line {
    ll a, b;
    ll get(ll x) { return a*x+b; }
  };
  struct Node {
    int l, r; ll s, e; Line line;
  };
  const ll INF = 4e18;
  const Line id = {0, INF};
  vector<Node> tree;
  void init(ll s, ll e) {
    tree.push_back({-1, -1, s, e, id});
  }
  void update(int node, Line v) {
    ll s = tree[node].s, e = tree[node].e, m = s+(e-s)/2;
    Line low = tree[node].line, high = v;
    if (low.get(s) > high.get(s)) swap(low, high);
    if (low.get(e) <= high.get(e)) {
      tree[node].line = low; return;
    }
    if (low.get(m) < high.get(m)) {
      tree[node].line = low;
      if (tree[node].r == -1) {
        tree[node].r = tree.size();
        tree.push_back({-1, -1, m+1, e, id});
      }
      update(tree[node].r, high);
    } else {
      tree[node].line = high;
      if (tree[node].l == -1) {
        tree[node].l = tree.size();
        tree.push_back({-1, -1, s, m, id});
      }
      update(tree[node].l, low);
    }
  }
  void update(int node, Line v, ll l, ll r) {
    if (r < tree[node].s || l > tree[node].e) return;
    if (l <= tree[node].s && tree[node].e <= r) {
      update(node, v); return;
    }
    ll s = tree[node].s, e = tree[node].e;
    ll m = s+(e-s)/2;
    if (tree[node].l == -1) {
      tree[node].l = tree.size();
      tree.push_back({-1, -1, s, m, id});
    }
    if (tree[node].r == -1) {
      tree[node].r = tree.size();
      tree.push_back({-1, -1, m+1, e, id});
    }
    update(tree[node].l, v, l, r);
    update(tree[node].r, v, l, r);
  }
  ll query(int node, ll x){
    if (node == -1) return INF;
    ll s = tree[node].s, e = tree[node].e;
    ll m = s + (e - s) / 2;
    if (x <= m) return min(tree[node].line.get(x),
    query(tree[node].l, x));
    else return min(tree[node].line.get(x),
    query(tree[node].r, x));
  }
};
```

3.7 DSU (Weighted)

```
struct DSU {
  vector<int> p; int comp;
  DSU(int n) : p(n+1, -1), comp(n) {}
  int find(int x) {
    return p[x] < 0 ? x : p[x] = find(p[x]);
  }
  bool merge(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return false;
    if (p[x] > p[y]) swap(x, y);
    p[x] += p[y]; p[y] = x; comp--;
    return true;
  }
  bool same(int x, int y) { return find(x) == find(y); }
  int size(int x) { return -p[find(x)]; }
};
struct WeightedDSU {
  vector<int> p, d;
  WeightedDSU(int n) : p(n+1), d(n+1, 0) { iota(all(p), 0); }
  int find(int x) {
    if (p[x] == x) return x;
    int r = find(p[x]); d[x] += d[p[x]];
    return p[x] = r;
  }
  int diff(int a, int b) { find(a); find(b); return
  d[a]-d[b]; } // for parity: ret &1
  bool merge(int a, int b, int w) { // a-b = w
    int pa = find(a), pb = find(b);
    if (pa == pb) return diff(a, b) == w;
    d[pa] = w + d[b]-d[a]; p[pa] = pb;
    return true;
  }
  bool same(int a, int b) { return find(a) == find(b); }
};
```

3.8 Trie

```
// Trie<10,0> tr;
template <int S = 26, char OS = 'a'> struct Trie {
  struct Node {
    int nxt[S], pass = 0, end = 0;
    Node() { memset(nxt, -1, sizeof(nxt)); }
  };
  vector<Node> tree;
  Trie() { init(); }
  void init() { tree.clear(); tree.emplace_back(); }
  // 원소 추가(1)/삭제(-1), O(|s|)
  void insert(const auto &s, int c = 1) {
    int u = 0; tree[u].pass += c;
    for (auto i : s) {
      int idx = i - OS;
      if (tree[u].nxt[idx] == -1) {
        tree[u].nxt[idx] = sz(tree); tree.emplace_back();
      }
      u = tree[u].nxt[idx]; tree[u].pass += c;
    }
    tree[u].end += c;
  }
  // 원소 삭제, O(|s|)
  bool erase(const auto& s, int c = 1) {
    if (count(s) < c) return false;
    insert(s, -c); return true;
  }
  int walk(const auto& s) const {
    int u = 0;
    for (auto i : s) {
      int v = tree[u].nxt[i-OS];
      if (v == -1) return -1;
      u = v;
    } return u;
  }
```

```
  int count(const auto& s) const { // 완전 일치 개수, O(|s|)
    int u = walk(s);
    return u == -1 ? 0 : tree[u].end;
  }
  int count_prefix(const auto& s) const { // prefix로 포함
  개수, O(|s|)
    int u = walk(s);
    return u == -1 ? 0 : tree[u].pass;
  }
};
template <typename T = ll, int B = 62> struct XorTrie {
  struct Node {
    int nxt[2]{ -1, -1 }, cnt = 0; // 서브트리 원소개수
  };
  vector<Node> tree;
  XorTrie() { init(); }
  void init() { tree.clear(); tree.emplace_back(); }
  // 원소 추가(1)/삭제(-1), O(B)
  void insert(T x, int c = 1) {
    int u = 0; tree[u].cnt += c;
    for (int i = B-1; i >= 0; i--) {
      int b = (x >> i) & 1;
      if (tree[u].nxt[b] == -1) {
        tree[u].nxt[b] = sz(tree); tree.emplace_back();
      }
      u = tree[u].nxt[b]; tree[u].cnt += c;
    }
  }
  // 원소 삭제, O(B)
  bool erase(T x, int c = 1) {
    if (count(x) < c) return false;
    insert(x, -c); return true;
  }
  // x 개수 조회, O(B)
  int count(T x) const {
    int u = 0;
    for (int i = B-1; i >= 0; i--) {
      int b = (x >> i) & 1;
      if (u == -1 || tree[u].nxt[b] == -1) return 0;
      u = tree[u].nxt[b];
    }
    return tree[u].cnt;
  }
  // x^y 기준 k번째 최솟값(1-idx), O(B)
  T kth_xor(T x, int k) const {
    if (k <= 0 || k > tree[0].cnt) return -1;
    int u = 0; T res = 0;
    for (int i = B-1; i >= 0; i--) {
      int b = (x >> i) & 1, l = tree[u].nxt[b], l_cnt = (l
      != -1 ? tree[l].cnt : 0);
      if (k <= l_cnt) {
        u = l;
      } else {
        k -= l_cnt; res |= (T(1) << i);
        u = tree[u].nxt[b ^ 1];
      }
    }
    return res;
  }
  T min_xor(T x) const { return kth_xor(x, 1); }
  T max_xor(T x) const { return kth_xor(x, tree[0].cnt); }
  // (x^y) < k 만족하는 y 개수, O(B)
  int count_less(T x, T k) const {
    int u = 0, res = 0;
    for (int i = B-1; i >= 0; i--) {
      if (u == -1) break;
      int bx = (x >> i) & 1, bk = (k >> i) & 1;
      if (bk) {
        int same = tree[u].nxt[bx];
        if (same != -1) res += tree[same].cnt;
        u = tree[u].nxt[bx ^ 1];
      } else {
        u = tree[u].nxt[bx];
      }
    }
    return res;
  }
};
```

3.9 Disjoint Sparse Table

Usage: 결합법칙만 필요하며 멱등성은 불필요

```
template <typename T, typename Op>
struct SparseTable {
  int n, lg; Op f;
  vector<vector<T>> st;
  SparseTable(const vector<T>& v, Op op) : n(sz(v)), f(op) {
    lg = __lg(max(1, n)) + 1;
    st.assign(lg, vector<T>(n)); st[0] = v;
    for (int i = 1; i < lg; i++) {
      int len = 1 << i;
      for (int m = len; m < n+len; m += len*2) {
        int e = min(m, n);
        st[i][e-1] = v[e-1];
        for (int j = e-2; j >= max(0, m-len); j--) st[i][j]
        = f(v[j], st[i][j+1]);
        if (m < n) {
          st[i][m] = v[m];
          for (int j = m+1; j < min(n, m+len); j++)
            st[i][j] = f(st[i][j-1], v[j]);
        }
      }
    }
  }
  T query(int l, int r) const { // [l, r], 0-idx
    if (l == r) return st[0][l];
    int k = __lg(l ^ r);
    return f(st[k][l], st[k][r]);
  }
};
// st(v, [](ll a, ll b){ return min(a, b); }); // RMQ
// st(v, [](ll a, ll b){ return gcd(a, b); }); // Range GCD
// st(v, [](ll a, ll b){ return a + b; }); // Range Sum
```

3.10 Mo's Algorithm

```
struct Query { int l, r, idx; };
vector<ll> mos(vector<int> a, vector<Query> q) {
  int n = sz(a);
  auto tmp = a; compress(tmp);
  for (auto &x : a) x = lower_bound(all(tmp), x) -
  tmp.begin();
  int k = *max_element(all(a)) + 1;
  int B = max(1, (int)(n / sqrt(max(1, sz(q)))));
  sort(all(q), [&](const Query& x, const Query& y) {
    int bx = x.l / B, by = y.l / B;
    if (bx != by) return bx < by;
    return (bx & 1) ? x.r > y.r : x.r < y.r;
  });
  vector<ll> cnt(k), res(sz(q));
  ll now = 0;
  auto add = [&](int i) { if (cnt[a[i]]++ == 0) now++; };
  auto remove = [&](int i) { if (--cnt[a[i]] == 0) now--; };
  int l = 0, r = 0; // 0-idx, [l, r)
  for (auto &[ql, qr, idx] : q) {
    while (l > ql) add(--l);
    while (r < qr) add(r++);
    while (l < ql) remove(l++);
    while (r > qr) remove(--r);
    res[idx] = now;
  }
  return res;
}
```

4 Graph

4.1 Bellman Ford

Time Complexity: $O(VE)$

```
auto bellman = [&](int s) -> bool {
  fill(all(d), INF); d[s] = 0; bool chk = 0;
  for (int i = 0; i < n; i++) { chk = 0;
    for (int u = 1; u <= n; u++) {
      if (d[u] == INF) continue;
      for (auto [w, v] : adj[u]) if (d[v] > d[u] + w) {
        d[v] = d[u] + w; chk = 1; if (i == n - 1) return 0;
      }
    }
    if (!chk) break;
  }
  return 1;
};
```

4.2 Eulerian Path

Time Complexity: $O(V + E)$

```
// 모든 간선을 한 번씩 사용하는 경로·사이클 복원. 정점
1-based.
// 무방향: 홀수 차수 2개면 그중 하나, 0개면 간선 있는 정점에서
시작.
// 방향: out-in이 +1,-1인 정점 각각 1개, 나머지 0이면 +1에서
시작.
//       모두 0이면 나가는 간선 있는 정점에서 시작. 그 외는
불가능.
// path(s): 정점 방문 순서. 미방문 간선이 남으면 {}. adj는
소모됨.
struct Euler {
  int m = 0; bool dir;
```

```
  vector<vector<pair<int,int>>> adj;
  Euler(int n, bool dir = false) : dir(dir), adj(n+1) {}
  void add(int u, int v) {
    adj[u].push_back({ v, m });
    if (!dir) adj[v].push_back({ u, m });
    m++;
  }
  vector<int> path(int s) {
    vector<int> st{s}, res;
    vector<bool> used(m);
    while (!st.empty()) {
      int u = st.back();
      if (adj[u].empty()) res.push_back(u), st.pop_back();
      else {
        auto [v, id] = adj[u].back(); adj[u].pop_back();
        if (!used[id]) used[id] = true, st.push_back(v);
      }
    }
    if (sz(res) != m+1) return {};
    reverse(all(res)); return res;
  }
};
```

4.3 Offline Dynamic Connectivity

```
// 정점 1-based, 시간 0-based.
struct ODC {
  int q, comp; vector<int> p;
  vector<pair<int,int>> hist;
  vector<vector<pair<int,int>>> seg;
  ODC(int n, int q) : q(q), comp(n), p(n+1, -1), seg(4*q+5) {}
  int find(int x) { return p[x] < 0 ? x : find(p[x]); }
  bool same(int x, int y) { return find(x) == find(y); }
  int size(int x) { return -p[find(x)]; }
  void merge(int x, int y) {
    x = find(x); y = find(y);
    if (x == y) return;
    if (p[x] > p[y]) swap(x, y);
    hist.push_back({ y, p[y] });
    p[x] += p[y]; p[y] = x; comp--;
  }
  void rollback(int t) {
    while (sz(hist) > t) {
      auto [y, v] = hist.back(); hist.pop_back();
      p[p[y]] -= v; p[y] = v; comp++;
    }
  }
  void add(int nd, int s, int e, int l, int r, int u, int v) {
    if (r <= s || e <= l) return;
    if (l <= s && e <= r) return void(seg[nd].push_back({ u,
    v }));
    int m = (s+e)>>1;
    add(nd<<1, s, m, l, r, u, v);
    add(nd<<1|1, m, e, l, r, u, v);
  }
  void add(int l, int r, int u, int v) { // [l,r) 동안
  존재하는 간선
    if (l < r) add(1, 0, q, l, r, u, v);
  }
  void dfs(int nd, int s, int e, auto &query) {
    int t = sz(hist);
    for (auto [u, v] : seg[nd]) merge(u, v);
    if (e-s == 1) query(s);
    else {
      int m = (s+e)>>1;
      dfs(nd<<1, s, m, query); dfs(nd<<1|1, m, e, query);
    }
    rollback(t);
  }
  // 각 시각 t에서 query(t) 호출
  void solve(auto query) { if (q) dfs(1, 0, q, query); }
};
ODC dc(3, 5);
dc.add(0, 3, 1, 2); dc.add(1, 5, 2, 3);
dc.solve([&](int t) {
  if (t==2 || t==4) cout << dc.same(1, 3);
}); // 출력: 1, 0
```

4.4 Euler Tour / Subtree Flattening

```
struct EulerTour {
  vector<vector<int>> adj; int n, pv = 0;
  vector<int> in, out, rev, dep;
  EulerTour(int n) : n(n), adj(n+1), in(n+1), out(n+1),
  rev(n+1), dep(n+1) {}
  void add(int u, int v) { adj[u].push_back(v);
  adj[v].push_back(u); }
  void dfs(int u, int p = 0, int d = 0) {
    in[u] = ++pv; rev[pv] = u; dep[u] = d;
    for (auto v : adj[u]) if (v != p) dfs(v, u, d+1);
    out[u] = pv;
  }
  bool chk(int u, int v) { // u가 v의 조상(또는 본인)인지 판별
    return in[u] <= in[v] && out[v] <= out[u];
  }
  int size(int u) { return out[u]-in[u]+1; }
};
// EulerTour et(N); et.dfs(root); / u의 서브트리 : [in[u],
out[u]]
```

4.5 HLD

Usage: for path queries on trees.
Time Complexity: $O(\log^2 N)$

```
// HLD: 트리 경로/서브트리 쿼리를 세그트리 구간으로 변환
(1-idx 정점).
// hld.build(a) // a[u]=정점 u 값 (1-indexed). 정점/간선
가중치 모두 지원
// path_upd(u,v,x, edge=0) // 경로 u~v에 x 적용 (edge=1이면
LCA 정점 제외, 간선용)
// setv(u,x)/addv(u,x) // 정점 단일 갱신
// qsum/qmin/qmax(u,v, edge=0) // 경로 쿼리
// * edge=true: 간선 가중치 문제에서 씀 (정점을 간선에 대응,
LCA 빼고)
/* Data Structure/Segment Tree */
template<typename T = ll> struct HLD {
  int n, pv;
  vector<vector<int>> adj;
  vector<int> siz, dep, par, top, in;
  SegTree<T> seg;
  HLD(int n) : n(n), adj(n+1), siz(n+1), dep(n+1), par(n+1),
  top(n+1), in(n+1), seg(n) {}
  void add(int u, int v) { adj[u].push_back(v);
  adj[v].push_back(u); }
  void dfs1(int u, int p) {
    siz[u] = 1; dep[u] = dep[p]+1; par[u] = p;
    for (auto& v : adj[u]) {
      if (v == p) continue;
      dfs1(v, u); siz[u] += siz[v];
      if (adj[u][0] == p || siz[v] > siz[adj[u][0]]) swap(v,
      adj[u][0]);
    }
  }
  void dfs2(int u, int p) {
    in[u] = pv++;
    for (auto v : adj[u]) {
      if (v == p) continue;
      top[v] = (v == adj[u][0] ? top[u] : v);
      dfs2(v, u);
    }
  }
  void build(const vector<T>& a = {}) { // a[u]: vertex
  value, 1-indexed
    pv = 0;
    for (int i = 1; i <= n; i++) {
      if (!siz[i]) {
        top[i] = i;
        dfs1(i, 0);
        dfs2(i, 0);
      }
    }
    vector<T> base(n);
    if (sz(a)) for (int i = 1; i <= n; i++) base[in[i]] =
    a[i];
    seg.build(base);
  }
  int lca(int u, int v) {
    while (top[u] != top[v]) {
      if (dep[top[u]] < dep[top[v]]) swap(u, v);
      u = par[top[u]];
    }
    return dep[u] < dep[v] ? u : v;
  }
  int path_len(int u, int v) {
    return dep[u] + dep[v] - 2*dep[lca(u, v)] + 1;
  }
  void path_upd(int u, int v, T x, bool edge = false) {
    while (top[u] != top[v]) {
      if (dep[top[u]] < dep[top[v]]) swap(u, v);
      seg.add(in[top[u]], in[u], x); // or seg.set
      u = par[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    if (in[u] + edge <= in[v]) {
      seg.add(in[u] + edge, in[v], x); // or seg.set
    }
  }
```

```
  void setv(int u, T x) { seg.set(in[u], in[u], x); }
  void addv(int u, T x) { seg.add(in[u], in[u], x); }
  typename SegTree<T>::Nd path_qry(int u, int v, bool edge =
  false) {
    typename SegTree<T>::Nd res = seg.id();
    while (top[u] != top[v]) {
      if (dep[top[u]] < dep[top[v]]) swap(u, v);
      res = seg.merge(res, seg.query(in[top[u]], in[u]));
      u = par[top[u]];
    }
    if (dep[u] > dep[v]) swap(u, v);
    return seg.merge(res, seg.query(in[u]+edge, in[v]));
  }
  T qsum(int u, int v, bool edge = false) { return
  path_qry(u, v, edge).sum; }
  T qmin(int u, int v, bool edge = false) { return
  path_qry(u, v, edge).mn; }
  T qmax(int u, int v, bool edge = false) { return
  path_qry(u, v, edge).mx; }
};
```

4.6 Virtual Tree

```
// 쿼리마다 주어진 K개 정점 사이의 거리 합·최소 연결
서브트리·트리 DP 등.
// 전체 트리 대신 선택 정점과 필요한 LCA만 남겨 O(K)개
정점에서 처리.
// HLD 의존. 하나의 트리에서 hld.build() 이후 사용.
// nodes: 선택 정점 + 필요한 LCA, DFS 순서.
// edges: {부모, 자식}, 원래 정점 번호.
// build(): 가상 트리 루트 반환. 빈 집합이면 -1.
struct VirtualTree {
  vector<int> nodes;
  vector<pair<int,int>> edges;
  int build(vector<int> v, auto& hld) {
    nodes.clear(); edges.clear();
    if (v.empty()) return -1;
    auto cmp = [&](int a, int b) { return hld.in[a] <
    hld.in[b]; };
    sort(all(v), cmp); v.erase(unique(all(v)), v.end());
    int k = sz(v);
    for (int i = 1; i < k; i++) v.push_back(hld.lca(v[i-1],
    v[i]));
    sort(all(v), cmp); v.erase(unique(all(v)), v.end());
    nodes = v; vector<int> st;
    for (int u : nodes) {
      while (!st.empty() && !(hld.in[st.back()] <= hld.in[u]
      && hld.in[u] < hld.in[st.back()] +
      hld.siz[st.back()])) st.pop_back();
      if (!st.empty()) edges.push_back({ st.back(), u });
      st.push_back(u);
    }
    return nodes[0];
  }
};
VirtualTree vt;
int root = vt.build(v, hld); // v: 선택한 정점 목록
for (auto [p, u] : vt.edges) {
  int len = hld.dep[u] - hld.dep[p]; // 압축된 경로의 간선 수
  // p -> u 처리
}
```

4.7 Centroid Decomposition

Usage: for path/distance problems.
Time Complexity: $O(N \log N)$ build

```
struct CentroidTree {
  vector<vector<int>> adj, c_adj; // adj: 원본트리 / c_adj:
  센트로이드트리
  vector<int> sz, par, vis; int N;
  CentroidTree(int n) : N(n), adj(n+1), c_adj(n+1), sz(n+1),
  par(n+1), vis(n+1) {}
  void add_edge(int u, int v) { adj[u].push_back(v);
  adj[v].push_back(u); }
  int get_sz(int curr, int prev) {
    sz[curr] = 1;
    for (int next : adj[curr])
      if (next != prev && !vis[next]) sz[curr] +=
      get_sz(next, curr);
    return sz[curr];
  }
  int get_cent(int curr, int prev, int to_sz) {
    for (int next : adj[curr])
      if (next != prev && !vis[next] && sz[next] > to_sz / 2)
        return get_cent(next, curr, to_sz);
    return curr;
  }
  void solve(int u) {
    // * solve(u): 현재 centroid u를 지나는 모든 경로를 여기서
    계산.
    //  - u에서 각 서브트리로 DFS하며 거리/조건 집계
    //  - 서로 다른 서브트리에서 오는 경로끼리 쌍을 만들 때
    중복 주의
    // build(1) 호출하면 자동으로 centroid tree 완성.
  }
  int build(int curr, int p = -1) {
    int cent = get_cent(curr, -1, get_sz(curr, -1));
    solve(cent);
    vis[cent] = 1; par[cent] = p;
    for (int next : adj[cent]) {
      if (!vis[next]) {
        int child = build(next, cent);
        c_adj[cent].push_back(child); // 센트로이드 계층 연결
      }
    }
    return cent;
  }
};
```

4.8 Tree Isomorphism

```
struct TreeIso {
  map<vector<int>,int> mp;
  int encode(const vector<vector<int>> &g, int root = 0) {
    int n = sz(g)-1;
    function<int(int,int)> dfs = [&](int u, int p) {
      vector<int> v;
      for (int x : g[u]) if (x != p) v.push_back(dfs(x, u));
      sort(all(v));
      return mp.emplace(v, sz(mp)+1).first->second;
    };
    if (root) return dfs(root, 0);
    vector<int> c;
    function<int(int,int)> cent = [&](int u, int p) {
      int s = 1, mx = 0;
      for (int x : g[u]) if (x != p) {
        int t = cent(x, u);
        s += t; mx = max(mx, t);
      }
      if (max(mx, n-s) <= n/2) c.push_back(u);
      return s;
    };
    cent(1, 0); int ret = 2e9;
    for (int x : c) ret = min(ret, dfs(x, 0));
    return ret;
  }
} ti;
// ti.encode(a) == ti.encode(b);
// ti.encode(a, ra) == ti.encode(b, rb); // 루트 지정
```

4.9 Bipartite Matching

Time Complexity: $O(E\sqrt{V})$

```
struct BiMatch { // Hopcroft-Karp
  vector<vector<int>> graph, grev;
  vector<int> mA, mB, dist, work;
  vector<bool> visA, visB, fA, fB; // vertex i can be
  excluded from some max matching
  int ns, ms;
  BiMatch(int n, int m) : ns(n), ms(m), graph(n+1),
  grev(m+1), mA(n+1), mB(m+1), dist(n+1), work(n+1) {}
  void add(int a, int b) { graph[a].push_back(b);
  grev[b].push_back(a); }
  void bfs() {
    fill(all(dist), -1); queue<int> q;
    for (int i = 1; i <= ns; i++) if (!mA[i]) {
      dist[i] = 0; q.push(i);
    }
    while (!q.empty()) {
      int i = q.front(); q.pop();
      for (auto j : graph[i]) {
        int k = mB[j];
        if (k && dist[k] == -1) {
          dist[k] = dist[i] + 1; q.push(k);
        }
      }
    }
  }
  bool dfs(int cur) {
    for (int& i = work[cur]; i < sz(graph[cur]); i++) {
      int nb = graph[cur][i], ori = mB[nb];
      if (!ori || dist[ori] == dist[cur] + 1 && dfs(ori)) {
        mA[cur] = nb; mB[nb] = cur; return true;
      }
    }
    return false;
  }
```

```
int match() {
  int ans = 0;
  for (int i = 1; i <= ns; i++) for (int j : graph[i]) {
    if (!mB[j]) {
      mA[i] = j; mB[j] = i; ans++; break;
    }
  }
  while (1) {
    fill(all(work), 0); bfs(); int cnt = 0;
    for (int i = 1; i <= ns; i++) {
      if (!mA[i] && dfs(i)) cnt++;
    }
    if (!cnt) break;
    ans += cnt;
  }
  return ans;
}
void chkEss() {
  fA.assign(ns+1, 0); fB.assign(ms+1, 0);
  visA.assign(ns+1, 0); visB.assign(ms+1, 0);
  queue<int> q;
  for (int i = 1; i <= ns; i++) if (!mA[i]) {
    fA[i] = visA[i] = 1; q.push(i);
  }
  while (!q.empty()) {
    int u = q.front(); q.pop();
    for (int v : graph[u]) if (!visB[v]) {
      visB[v] = 1; int r = mB[v];
      if (r && !fA[r]) {
        fA[r] = visA[r] = 1; q.push(r);
      }
    }
  }
  for (int i = 1; i <= ms; i++) if (!mB[i]) {
    fB[i] = 1; q.push(i);
  }
  while (!q.empty()) {
    int v = q.front(); q.pop();
    for (int u : grev[v]) {
      int r = mA[u];
      if (r && !fB[r]) {
        fB[r] = 1; q.push(r);
      }
    }
  }
}
pair<vector<int>,vector<int>> vertex() { // find minimum
vertex cover
  chkEss(); vector<int> va, vb;
  for (int i = 1; i <= ns; i++) if (!visA[i])
  va.push_back(i);
  for (int i = 1; i <= ms; i++) if (visB[i])
  vb.push_back(i);
  return { va, vb };
}
// 미매칭인 왼쪽 정점 u에서 한 번 증강. 성공하면 크기++
// for (int u = 1; u <= bm.ns; u++) cnt += bm.augment(u);
bool augment(int u) {
  if (mA[u]) return false;
  vector<char> vis(ns+1);
  auto dfs = [&](auto&& self, int u) -> bool {
    if (vis[u]) return false;
    vis[u] = 1;
    for (int v : graph[u]) {
      if (!mB[v] || self(self, mB[v])) {
        mA[u] = v; mB[v] = u; return true;
      }
    }
    return false;
  };
  return dfs(dfs, u);
}
};
```

4.10 Hungarian

Usage: 비용 행렬 기반 일대일 최소 비용 매칭. 용량·네트워크 제약이 추가되면 MCMF

Time Complexity: $O(N^3)$

```
template<typename T = ll> struct Hungarian {
  vector<T> u, v, ans; vector<int> p, way;
  int n, m; const T INF = 1e15;
  Hungarian(int n, int m) : n(n), m(m), u(n+1), v(m+1),
  p(m+1), way(m+1) {}
  T solve (const vector<vector<T>> &a) {
    ans.clear();
    vector<T> mv(m+1); vector<bool> vis(m+1);
    for (int i = 1; i <= n; i++) {
      fill(all(mv), INF); fill(all(vis), false);
      p[0] = i; int j0 = 0;
      while (p[j0] != 0) {
        vis[j0] = true; T dt = INF;
        int i0 = p[j0], j1 = 0;
        for (int j = 1; j <= m; j++) {
          if (vis[j]) continue;
          T cur = a[i0][j] - u[i0] - v[j];
          if (cur < mv[j]) mv[j] = cur, way[j] = j0;
          if (mv[j] < dt) dt = mv[j], j1 = j;
        }
        for (int j = 0; j <= m; j++) {
          if (vis[j]) u[p[j]] += dt, v[j] -= dt;
          else mv[j] -= dt;
        }
        j0 = j1;
      }
      while (j0 != 0) {
        int j1 = way[j0];
        p[j0] = p[j1]; j0 = j1;
      }
      ans.push_back(-v[0]);
    }
    return sz(ans) ? ans.back() : 0;
  }
  T cost(int k) { return ans[k-1]; }
  vector<int> match() {
    vector<int> res(n+1);
    for (int i = 1; i <= m; i++) if (p[i]) res[p[i]] = i;
    return res;
  }
};
```

4.11 Dinic

Time Complexity: $O(V^2E)$

```
const ll INF = 1e18;
struct Dinic {
  struct Edge { int to; ll cap; int rev; };
  vector<vector<Edge>> graph;
  vector<int> level, work; int n;
  Dinic(int n) : n(n), graph(n+1), level(n+1), work(n+1) {}
  void add(int u, int v, ll cap) {
    graph[u].push_back({ v, cap, sz(graph[v])});
    graph[v].push_back({ u, 0, sz(graph[u])-1});
  }
  bool bfs(int s, int t) {
    fill(all(level), -1); level[s] = 0;
    queue<int> q; q.push(s);
    while (!q.empty()) {
      int cur = q.front(); q.pop();
      for (auto [nxt, cap, rev] : graph[cur]) {
        if (cap > 0 && level[nxt] == -1) {
          level[nxt] = level[cur]+1;
          if (nxt == t) return true;
          q.push(nxt);
        }
      }
    }
    return (level[t] != -1);
  }
  ll dfs(int cur, int t, ll flow) {
    if (cur == t) return flow;
    for (int& i = work[cur]; i < sz(graph[cur]); i++) {
      auto& [nxt, cap, rev] = graph[cur][i];
      if (cap > 0 && level[nxt] == level[cur]+1) {
        ll push = dfs(nxt, t, min(flow, cap));
        if (push > 0) {
          cap -= push; graph[nxt][rev].cap += push;
          return push;
        }
      }
    }
    return 0;
  }
  ll flow(int s, int t) {
    ll ans = 0;
    while (bfs(s, t)) {
      fill(all(work), 0);
      while (auto flow = dfs(s, t, INF)) ans += flow;
    }
    return ans;
  }
  vector<bool> mincut(int s) {
    vector<bool> vis(n+1); vis[s] = true;
    queue<int> q; q.push(s);
    while (!q.empty()) {
      int cur = q.front(); q.pop();
```

```
      for (auto [nxt, cap, rev] : graph[cur]) {
        if (cap > 0 && !vis[nxt]) {
          vis[nxt] = true; q.push(nxt);
        }
      }
    }
    return vis;
  }
};
```

4.12 MCMF

Time Complexity: SPFA 1회: 최악 $O(VE)$, 전체: $O(AVE)$ (A: SPFA 실행 횟수)

```
template<typename T = ll>
struct MCMF {
  struct Edge { int to, rev; ll cap; T cost; };
  const T INF = 1e18, EPS = 1e-9;
  vector<vector<Edge>> graph; vector<T> dist;
  vector<int> ptr; vector<bool> vis; int n;
  MCMF(int n) : n(n), graph(n+1), dist(n+1), ptr(n+1),
  vis(n+1) {}
  void add(int u, int v, ll cap, T cost) {
    graph[u].push_back({ v, sz(graph[v]), cap, cost });
    graph[v].push_back({ u, sz(graph[u])-1, 0, -cost });
  }
  bool spfa(int s, int t) {
    fill(all(dist), INF); fill(all(vis), 0);
    deque<int> q; q.push_back(s); dist[s] = 0; vis[s] = 1;
    while (!q.empty()) {
      if (dist[q.front()] > dist[q.back()]) swap(q.front(),
      q.back());
      int cur = q.front(); q.pop_front(); vis[cur] = 0;
      for (auto& [nxt, rev, cap, cost] : graph[cur]) {
        if (cap > 0 && dist[nxt] > dist[cur] + cost + EPS) {
          dist[nxt] = dist[cur] + cost;
          if (!vis[nxt]) {
            vis[nxt] = 1;
            if (!q.empty() && dist[nxt] < dist[q.front()])
            q.push_front(nxt);
            else q.push_back(nxt);
          }
        }
      }
    }
    return dist[t] < INF;
  }
  ll dfs(int cur, int t, ll flow, T& cost) {
    if (cur == t) return flow;
    vis[cur] = 1;
    for (int& i = ptr[cur]; i < sz(graph[cur]); i++) {
      auto& [nxt, rev, cap, cst] = graph[cur][i];
      if (!vis[nxt] && cap > 0 && abs(dist[nxt] - (dist[cur]
      + cst)) <= EPS) {
        ll push = dfs(nxt, t, min(flow, cap), cost);
        if (push > 0) {
          graph[nxt][rev].cap += push;
          cost += push * cst; cap -= push;
          vis[cur] = 0; return push;
        }
      }
    }
    return 0;
  }
  pair<ll,T> flow(int s, int t) {
    ll res = 0; T cost = 0;
    while (spfa(s, t)) {
      // if (dist[t] >= -EPS) break; // Min-Cost Flow
      fill(all(ptr), 0); fill(all(vis), 0);
      while (1) {
        ll push = dfs(s, t, 1e18, cost);
        if (push == 0) break;
        res += push;
      }
    }
    return { res, cost };
  }
};
```

4.13 Circulation

Usage: Flow with lower and upper bounds.

```
const ll INF = 1e18;
struct Dinic {}; // or MCMF
struct Circulation {
  vector<ll> demand, low; vector<pair<int,int>> edge;
  int n, S, T; Dinic dn;
  Circulation(int n) : n(n), S(n+1), T(n+2), demand(n+3, 0),
  dn(n+2) {};
  void add_demand(int u, ll d) { demand[u] += d; }
  int add(int u, int v, ll l, ll r) {
    demand[u] -= l; demand[v] += l;
    dn.add(u, v, r - l); low.push_back(l);
    edge.push_back({ u, sz(dn.graph[u])-1 });
    return sz(edge)-1;
  }
  ll solve() {
    ll sum = 0, res = 0;
    for (int i = 0; i <= n; i++) sum += demand[i];
    if (sum != 0) return -1;
    for (int i = 0; i <= n; i++) {
      if (demand[i] > 0) {
        dn.add(S, i, demand[i]); res += demand[i];
      }
      else if (demand[i] < 0) dn.add(i, T, -demand[i]);
    }
    ll f = dn.flow(S, T);
    return (f != res ? -1 : f);
  }
  ll get_flow(int i) { // get actual flow
    auto [u, idx] = edge[i];
    int v = dn.graph[u][idx].to, rev = dn.graph[u][idx].rev;
    return dn.graph[v][rev].cap + low[i];
  }
  // min flow: int t = cc.add(T, S, 0, INF); cc.solve();
  cc.cut(t); cc.dn.flow(N, 1);
  void cut(int i) {
    auto [u, idx] = edge[i];
    auto& fwd = dn.graph[u][idx];
    auto& bwd = dn.graph[fwd.to][fwd.rev];
    fwd.cap = bwd.cap = 0;
  }
};
```

4.14 SCC

Usage: 방향 그래프의 순환 의존성을 압축하여 DAG로 변환

Time Complexity: $O(V + E)$

```
// 0-based. id는 역위상순: 서로 다른 SCC 사이의 u->v이면
id[u]>id[v].
struct SCC {
  int n, cnt = 0, t = 0;
  vector<vector<int>> adj;
  vector<int> dfn, low, id, st;
  SCC(int n) : n(n), adj(n), dfn(n), low(n), id(n) {}
  void add(int u, int v) { adj[u].push_back(v); }
  int add() { // 정점 추가, 새 정점 번호 반환
    adj.emplace_back(); dfn.emplace_back();
    low.emplace_back(); id.emplace_back(); return n++;
  }
  void dfs(int u) {
    dfn[u] = low[u] = t++; st.push_back(u);
    for (int v : adj[u]) {
      if (dfn[v] == -1) dfs(v), low[u] = min(low[u], low[v]);
      else if (id[v] == -1) low[u] = min(low[u], dfn[v]);
    }
    if (low[u] == dfn[u]) {
      while (1) {
        int v = st.back(); st.pop_back(); id[v] = cnt;
        if (u == v) break;
      }
      cnt++;
    }
  }
  void build() { // 간선 추가 후 재호출 가능
    cnt = t = 0; st.clear();
    fill(all(dfn), -1); fill(all(id), -1);
    for (int i = 0; i < n; i++) if (dfn[i] == -1) dfs(i);
  }
};
```

4.15 2-sat

Usage: Solves 2-SAT in $O(V + E)$; x_i is true if $SCC(x_i) < SCC(\neg x_i)$.

Time Complexity: $O(V + E)$

```
// 변수 0-based. 리터럴 2*i: x_i, 2*i+1: !x_i.
struct TwoSat {
  int n; SCC sc; vector<bool> ans;
  TwoSat(int n) : n(n), sc(2*n) {}
  int I(int x) { return x^1; }
  void add_edge(int u, int v) { // u->v와 대우
    sc.add(u, v); sc.add(I(v), I(u));
  }
  void add_clause(int u, int v) { add_edge(I(u), v); }
  void add_clause(int u, bool ut, int v, bool vt) {
    add_clause(u*2 + !ut, v*2 + !vt); // (u==ut) or (v==vt)
```

```
  }
  int add() { sc.add(); sc.add(); return n++; }
  void most_one(const vector<int>& v, bool cd = true) {
    int pre = -1;
    for (int i : v) {
      int x = i*2 + !cd, now = add()*2;
      add_edge(x, now);
      if (pre != -1) {
        add_edge(pre, now); add_edge(pre, I(x));
      }
      pre = now;
    }
  }
  bool solve() {
    sc.build(); ans.resize(n);
    for (int i = 0; i < n; i++) {
      if (sc.id[i*2] == sc.id[i*2+1]) return false;
      ans[i] = sc.id[i*2] < sc.id[i*2+1];
    }
    return true;
  }
};
```

4.16 BCC

Usage: 정점·간선 제거에 따른 연결성. 단절점·브리지·block-cut tree.
Time Complexity: $O(V+E)$

```
// 무방향 그래프, 정점 1-based. 중복 간선 허용, self-loop 제외.
// 정점 BCC: 단절점은 여러 성분에 속할 수 있음.
// build: O(V+E). 간선 추가 후 재호출 가능.
struct BCC {
  int n, t = 0, cnt = 0;
  vector<vector<int>> adj, bcc; // bcc[u]: u가 속한 성분 번호들
  vector<int> in, low, par, st;
  vector<char> cut; // cut[u]: 단절점 여부
  vector<pair<int,int>> bridge;
  BCC(int n) : n(n), adj(n+1), bcc(n+1), in(n+1),
    low(n+1), par(n+1), cut(n+1) {}
  void add(int u, int v) {
    adj[u].push_back(v); adj[v].push_back(u);
  }
  void dfs(int u, int p) {
    in[u] = low[u] = ++t; par[u] = p; st.push_back(u);
    int ch = 0; bool skip = false;
    for (int v : adj[u]) {
      // 부모 간선은 하나만 제외. 나머지 중복 간선은 역방향 간선.
      if (v == p && !skip) { skip = true; continue; }
      if (!in[v]) {
        ch++; dfs(v, u); low[u] = min(low[u], low[v]);
        if (low[v] >= in[u]) {
          if (p) cut[u] = 1;
          while (1) {
            int x = st.back(); st.pop_back();
            bcc[x].push_back(cnt);
            if (x == v) break;
          }
          bcc[u].push_back(cnt++);
        }
        if (low[v] > in[u])
          bridge.emplace_back(min(u,v), max(u,v));
      } else low[u] = min(low[u], in[v]);
    }
    if (!p) cut[u] = ch > 1;
  }
  // 단절점·브리지·BCC 소속 계산. 다른 조회 전에 호출.
  // 성분 번호 [0,cnt-1]. 고립 정점도 성분 하나로 처리.
  void build() {
    t = cnt = 0; st.clear(); bridge.clear();
    fill(all(in), 0); fill(all(cut), 0);
    for (auto& v : bcc) v.clear();
    for (int u = 1; u <= n; u++) if (!in[u]) {
      dfs(u, 0); st.pop_back();
      if (adj[u].empty()) bcc[u].push_back(cnt++);
    }
  }
  // 제거하면 연결 성분 수가 증가하는 정점들. O(V).
  vector<int> cutVertex() const {
    vector<int> res;
    for (int u = 1; u <= n; u++) if (cut[u]) res.push_back(u);
    return res;
  }
  // 제거하면 연결 성분 수가 증가하는 간선들. 순서는 미정렬.
  vector<pair<int,int>> cutEdge() const { return bridge; }
  // 정점 제거·단절점 경유 문제를 트리 경로 문제로 변환.
  // 원래 정점 [1,n], 성분 c의 정점 번호 n+c+1.
  // 비단절점도 유지하며, 원본이 비연결이면 forest.
  vector<vector<int>> block_cut() const {
    vector<vector<int>> g(n+cnt+1);
    for (int u = 1; u <= n; u++) for (int c : bcc[u]) {
      g[u].push_back(n+c+1); g[n+c+1].push_back(u);
    }
    return g;
  }
};
```

4.17 Push Relabel

Time Complexity: $O(V^3)$

```
template<typename T = ll> struct HLPP {
  struct Edge { int to; T cap; int rev; };
  T INF = numeric_limits<T>::max();
  vector<vector<Edge>> adj;
  vector<T> ex;
  vector<int> pos, lvl, nxt, gprv, gnxt;
  int n, high = 0, gap = 0, cnt = 0;
  HLPP(int n) : n(n+1), adj(n+2), ex(n+2), pos(n+2),
  lvl(n+2), nxt(2*n+3), gprv(2*n+3), gnxt(2*n+3) {}
  void add(int u, int v, T cap) {
    adj[u].push_back({ v, cap, sz(adj[v]) });
    adj[v].push_back({ u, 0, sz(adj[u])-1 });
  }
  void push(int u, int h) {
    nxt[u] = nxt[n+h]; nxt[n+h] = u;
    high = max(high, h);
  }
  void ugap(int u, int h) {
    gnxt[u] = gnxt[gprv[u] = n+h];
    gprv[gnxt[u]] = gnxt[gprv[u]] = u;
    gap = max(gap, h);
  }
  void dgap(int u) {
    gnxt[gprv[u]] = gnxt[u];
    gprv[gnxt[u]] = gprv[u];
  }
  void addex(int u, T f) {
    ex[u] += f;
    if (ex[u] == f) push(u, lvl[u]);
  }
  void upd(int u, int h) {
    if (lvl[u] != n+1) dgap(u);
    lvl[u] = h; if (h == n+1) return;
    ugap(u, h); if (ex[u] > 0) push(u, h);
  }
  void relabel(int t) {
    cnt = high = gap = 0;
    iota(nxt.begin()+n, nxt.end(), n);
    iota(gprv.begin()+n, gprv.end(), n);
    iota(gnxt.begin()+n, gnxt.end(), n);
    fill(all(lvl), n+1); fill(all(pos), 0);
    lvl[t] = 0; queue<int> q; q.push(t);
    while (!q.empty()) {
      int u = q.front(); q.pop();
      for (auto &e : adj[u]) {
        if (!adj[e.to][e.rev].cap || lvl[e.to] <= lvl[u]+1)
        continue;
        upd(e.to, lvl[u]+1); q.push(e.to);
      }
    }
  }
  void discharge(int u) {
    T &v = ex[u]; int h = n, Sz = sz(adj[u]);
    for (int &i = pos[u], k = Sz; k--; i = (i ? i : Sz)-1) {
      auto& e = adj[u][i];
      if (!e.cap) continue;
      if (lvl[u] != lvl[e.to]+1) {
        h = min(h, lvl[e.to]); continue;
      }
      auto f = min(v, e.cap);
      v -= f; addex(e.to, f); e.cap -= f;
      adj[e.to][e.rev].cap += f;
      if (!v) return;
    }
    cnt++;
    if (gnxt[gnxt[n+lvl[u]]] < n) {
      upd(u, h+1); return;
    }
    for (int i = lvl[u]; gap >= i; gap--) {
      while (gnxt[n+gap] < n) {
        int t = gnxt[n+gap];
        lvl[t] = n+1; dgap(t);
      }
    }
  }
```

```
  T flow(int s, int t) {
    relabel(t);
    addex(s, INF); ex[t] -= INF;
    while (~high) {
      int u = nxt[n+high];
      if (u >= n) { high--; continue; }
      nxt[n+high] = nxt[u];
      if (lvl[u] != high) continue;
      discharge(u);
      if (cnt >= 4*n) relabel(t);
    }
    return ex[t]+INF;
  }
};
```

4.18 Gomory Hu Tree

Usage: 무방향 용량 그래프의 여러 정점 쌍 최소컷. 트리 경로의 최소 간선 가중치로 조회

```
// Gomory-Hu Tree: 모든 정점 쌍의 최대 유량(Min-cut)을
표현하는 트리
// 트리에서 u-v 경로 상의 최소 가중치 간선 = 원본 그래프에서
u-v의 최대 유량
struct Dinic {};
struct Edge { int u, v; ll w; };
vector<Edge> gomory_hu(int n, const vector<Edge> &ev) {
  vector<Edge> tree;
  vector<int> par(n+1, 1);
  for (int i = 2; i <= n; i++) {
    Dinic dn(n);
    for (auto& [u, v, w] : ev) { dn.add(u, v, w); dn.add(v,
    u, w); }
    tree.push_back({i, par[i], dn.flow(i, par[i])});
    vector<bool> cut = dn.mincut(i);
    for (int j = i + 1; j <= n; j++) {
      if (cut[j] && par[j] == par[i]) par[j] = i;
    }
  }
  return tree;
}
// 인접 정점 간 최대 유량 합이 최대인 순열
auto res = gomory_hu(n, edges);
sort(all(res), [](const Edge& a, const Edge& b) { return a.w
> b.w; });
DSU uf(n); vector<vector<int>> p(n+1);
for (int i = 1; i <= n; i++) p[i] = {i};
ll ans = 0;
for (auto& e : res) {
  int u = uf.find(e.u), v = uf.find(e.v);
  if (!uf.merge(u, v)) continue;
  if (uf.find(u) == v) swap(u, v);
  p[u].insert(p[u].end(), all(p[v])); ans += e.w;
} // ans: 최대 합, p[uf.find(1)]: 해당 순열
```

4.19 General Matching

Time Complexity: $O(N^3)$

```
struct GeneralMatch {
  vector<vector<int>> adj;
  vector<int> vis, par, ori, mat, aux;
  queue<int> q; int n, t = 0;
  GeneralMatch(int n) : n(n), adj(n+1), vis(n+1), par(n+1),
  ori(n+1), mat(n+1), aux(n+1) {}
  void add(int a, int b) { adj[a].push_back(b);
  adj[b].push_back(a); }
  void augment(int v) {
    while (v) {
      int pv = par[v], nv = mat[pv];
      mat[v] = pv; mat[pv] = v; v = nv;
    }
  }
  int lca(int v, int w) {
    ++t; while (1) {
      if (v) {
        if (aux[v] == t) return v;
        aux[v] = t; v = ori[par[mat[v]]];
      }
      swap(v, w);
    }
  }
  void blossom(int v, int w, int a) {
    while (ori[v] != a) {
      par[v] = w; w = mat[v];
      if (vis[w] == 1) q.push(w), vis[w] = 0;
      ori[v] = ori[w] = a; v = par[w];
    }
  }
  bool bfs(int u, int ban = 0) {
    fill(all(vis), -1); iota(all(ori), 0);
    if (ban) vis[ban] = 2;
    q = queue<int>(); q.push(u); vis[u] = 0;
    while (!q.empty()) {
      int v = q.front(); q.pop();
      for (auto w : adj[v]) {
        if (vis[w] == -1) {
          par[w] = v; vis[w] = 1;
          if (!mat[w]) { augment(w); return true; }
          vis[mat[w]] = 0; q.push(mat[w]);
        }
        else if (vis[w] == 0 && ori[v] != ori[w]) {
          int a = lca(ori[v], ori[w]);
          blossom(w, v, a); blossom(v, w, a);
        }
      }
    }
    return false;
  }
  int match() {
    int ans = 0; vector<int> v(n); iota(all(v), 1);
    shuffle(all(v), rng);
    for (auto x : v) if (!mat[x]) {
      for (auto y : adj[x]) if (!mat[y]) {
        mat[x] = y, mat[y] = x;
        ans++; break;
      }
    }
    for (int i = 1; i <= n; i++) {
      if (!mat[i] && bfs(i)) ans++;
    }
    return ans;
  }
  bool chk(int u) { // find max matching except u
    if (!mat[u]) return true;
    auto tmp = mat; int v = mat[u];
    mat[u] = mat[v] = 0;
    bool res = bfs(v, u); mat = tmp;
    return res;
  }
};
```

5 DP

5.1 LIS / 3D LIS

```
// a[i] < a[j] -> lower_bound, a[i] <= a[j] -> upper_bound
vector<int> LIS(vector<int> v) {
  int n = sz(v);
  vector<int> lis, pos(n);
  for (int i = 0; i < n; i++) {
    auto it = lower_bound(all(lis), v[i]);
    pos[i] = it - lis.begin();
    if (it == lis.end()) lis.push_back(v[i]);
    else *it = v[i];
  }
  vector<int> res;
  for (int i = n-1, j = sz(lis)-1; i >= 0; i--) {
    if (pos[i] == j) {
      res.push_back(v[i]); j--;
    }
  }
  reverse(all(res));
  return res;
}
// 3D LIS
struct Cand {
  map<int,pair<int,int>> m; // y -> {최소 z, 원본 인덱스}
  int get(int y, int z) const {
    auto it = m.lower_bound(y); // non-strict: upper_bound(y)
    if (it == m.begin()) return -1;
    auto [pz, id] = prev(it)->second;
    return pz < z ? id : -1; // non-strict: pz <= z
  }
  void add(int y, int z, int id) {
    auto it = m.lower_bound(y);
    if (it != m.begin() && prev(it)->second.first <= z)
    return;
    while (it != m.end() && it->second.first >= z) it =
    m.erase(it);
    m.insert(it, { y, { z, id } });
  }
};
// 세 좌표가 모두 엄격히 증가하는 최장 체인(인덱스 반환).
vector<int> LIS3D(const vector<array<int,3>>& v) {
  int n = sz(v), last = -1;
  vector<int> ord(n), pre(n, -1); iota(all(ord), 0);
  sort(all(ord), [&](int i, int j) {
    // non-strict: return v[i] < v[j];
```

```
    if (v[i][0] != v[j][0]) return v[i][0] < v[j][0];
    return v[i][1] > v[j][1];
  });
  vector<Cand> res;
  for (int i : ord) {
    auto [x, y, z] = v[i];
    int lo = 0, hi = sz(res);
    while (lo < hi) {
      int mid = (lo+hi)/2;
      if (res[mid].get(y, z) != -1) lo = mid+1;
      else hi = mid;
    }
    if (lo) pre[i] = res[lo-1].get(y, z);
    if (lo == sz(res)) res.emplace_back(), last = i;
    res[lo].add(y, z, i);
  }
  vector<int> ans;
  for (int i = last; i != -1; i = pre[i]) ans.push_back(i);
  reverse(all(ans)); return ans;
}
```

5.2 Convex Hull Trick

Usage: dp[i] = max(dp[j] + b[j] * a[i])
Time Complexity: $O(N \log N)$

```
// Slopes(k) and queries(x) can be in any order
template <typename T> struct Line {
  mutable T k, m, p;
  bool operator<(const Line& o) const { return k < o.k; }
  bool operator<(T x) const { return p < x; }
};
template <typename T = ll> struct LineContainer :
multiset<Line<T>, less<>> {
  using iterator = typename multiset<Line<T>,
  less<>>::iterator;
  // (for doubles, use div(a,b) = a/b)
  static constexpr T inf = numeric_limits<T>::max();
  T div(T a, T b) { // floored division
    return a / b - ((a ^ b) < 0 && a % b); }
  bool isect(iterator x, iterator y) {
    if (y == this->end()) return x->p = inf, 0;
    if (x->k == y->k) x->p = x->m > y->m ? inf : -inf;
    else x->p = div(y->m - x->m, x->k - y->k);
    return x->p >= y->p;
  }
  void add(T k, T m) { // y = kx + m
    auto z = this->insert({k, m, 0}), y = z++, x = y;
    while (isect(y, z)) z = this->erase(z);
    if (x != this->begin() && isect(--x, y)) isect(x, y =
    this->erase(y));
    while ((y = x) != this->begin() && (--x)->p >= y->p)
    isect(x, this->erase(y));
  }
  T query(T x) {
    assert(!this->empty());
    auto l = *this->lower_bound(x);
    return l.k * x + l.m;
  }
}; // add(-k, -m), -query(x) for Lower hull(min)
LineContainer<ll> CHT;
```

```
dp[0] = 0; CHT.add(a[0], dp[0]);
for (int i = 1; i < n; i++) { // dp[i] = Max(a[j]*b[i] +
dp[j])
  dp[i] = CHT.query(b[i]);
  CHT.add(a[i], dp[i]);
}
```

5.3 Linear CHT

Usage: CHT when slopes(k)/queries(x) are monotonic.
Time Complexity: $O(N)$

```
using i128 = __int128;
// 기본: max, 기울기 a 단조증가, 쿼리 x 단조증가
// max: a↑ x↑ 그대로 / a↓ x↓ add의 교점 비교 양변 교환
// min: a↓ x↑ 그대로 / a↑ x↓ add의 교점 비교 양변 교환
// min은 추가로 같은 기울기 처리 >=를 <=로, query의 <=를 >=로
변경
// add/query 각각 amortized O(1)
struct PLL { ll x, y;
  PLL(const ll x = 0, const ll y = 0) : x(x), y(y) {}
  bool operator<= (const PLL& i) const {
    return (i128)x * i.y <= (i128)i.x * y; }
};
struct ConvexHull {
  static i128 F(const PLL& i, const ll x) { return (i128)i.x
  * x + i.y; }
  static PLL C(const PLL& a, const PLL& b) { return { a.y -
  b.y, b.x - a.x }; }
  deque<PLL> S;
  void add(const ll a, const ll b) {
    if (!S.empty() && S.back().x == a) {
      if (S.back().y >= b) return; // min: <=
      S.pop_back();
    }
    // max a↓ / min a↑: 아래 <=의 양변 교환
    while ( S.size() > 1 && C(S.back(), PLL(a, b)) <=
    C(S[S.size() - 2], S.back()) ) S.pop_back();
    S.push_back(PLL(a, b));
  }
  ll query(const ll x) {
    assert(!S.empty());
    while (S.size() > 1 && F(S[0], x) <= F(S[1], x)) // min:
    >=
      S.pop_front();
    return F(S[0], x);
  }
} CHT;
```

5.4 D&C optimization

Usage: dp[t][i] = min(dp[t-1][j] + c[j][i]), c is Monge. 이전 행에서 전이하는 DP의 최적 후보 위치가 비감소. 구간 분할 DP 등에 적용.
Time Complexity: $O(KN \log N)$

```
// dp[k][i] = min_j f(k,j,i). 이전 행을 먼저 계산해야 함.
// 조건: 최적 j(동률이면 가장 작은 j)가 i에 대해 비감소.
// 각 계산 대상 i에는 유효한 전이가 하나 이상 있어야 함.
// max: 초기값 INF -> -INF, 비교 < -> >, 불가능한 전이도 -INF.
// 상태/후보가 각각 O(N), f가 O(1)이면 한 행 O(N log N).
const ll INF = 1LL << 62;
ll dp[MAX_K][MAX_N];
ll f(int k, int j, int i); // 전이 값. 불가능하면 INF 반환.
void dnc(int k, int l, int r, int pL, int pR) {
  if (l > r) return;
  int opt = pL, mid = (l + r) / 2;
  dp[k][mid] = INF;
  for (int j = pL; j <= pR; j++) {
    ll val = f(k, j, mid);
    if (val < dp[k][mid]) {
      dp[k][mid] = val; opt = j;
    }
  }
  dnc(k, l, mid - 1, pL, opt);
  dnc(k, mid + 1, r, opt, pR);
}
// dp[k][i]: 앞의 i개 원소를 정확히 k개 구간으로 나눈 최소
비용
ll f(int k, int j, int i) {
  if (j >= i || dp[k-1][j] == INF) return INF;
  ll sum = S[i] - S[j];
  return dp[k-1][j] + sum * sum; // 계산 결과가 INF 미만이라고
  가정
}
// 1 <= K <= n. dp/S의 두 번째 차원은 n+1 이상.
for (int k = 0; k <= K; k++)
  fill(dp[k], dp[k] + n+1, INF);
dp[0][0] = 0;
for (int k = 1; k <= K; k++)
  dnc(k, k, n, k-1, n-1);
// 답: dp[K][n]
```

5.5 Knuth optimization

Time Complexity: $O(N^2)$

```
// 인접 구간을 합치는 최소 비용 DP. 예: 파일 합치기.
// dp[l][r]=min_k(dp[l][k]+dp[k+1][r])+cost(l,r), dp[i][i]=0.
// 조건: opt[l][r-1]<=opt[l][r]<=opt[l+1][r]. 동률이면 큰 k.
// 충분조건(a<=b<=c<=d): C(b,c)<=C(a,d),
C(a,c)+C(b,d)<=C(a,d)+C(b,c).
ll knuth(int n, auto cost) {
  const ll INF = 1LL << 62;
  vector<vector<ll>> dp(n, vector<ll>(n));
  vector<vector<int>> opt(n, vector<int>(n));
  for (int i = 0; i < n; i++) opt[i][i] = i;
  for (int len = 2; len <= n; len++)
    for (int l = 0, r = len-1; r < n; l++, r++) {
      dp[l][r] = INF; ll c = cost(l, r);
      for (int k = opt[l][r-1]; k <= min(r-1, opt[l+1][r]);
      k++) {
        ll v = dp[l][k] + dp[k+1][r] + c;
        if (v <= dp[l][r]) dp[l][r] = v, opt[l][r] = k;
      }
    }
  return dp[0][n-1];
}
vector<ll> a{40, 30, 30, 50}, S(sz(a)+1); // 파일합치기
for (int i = 0; i < sz(a); i++) S[i+1] = S[i]+a[i];
ll ans = knuth(sz(a), [&](int l, int r) {
  return S[r+1]-S[l];
}); // 300
```

5.6 Monotone Queue optimization

Usage: dp[i] = min(dp[j] + c[j][i]), c is Monge, find cross. 같은 행의 앞선 상태에서 전이하며, 후보의 우열이 한 번만 역전. 후보별 최적 구간을 deque로 관리.
Time Complexity: $O(N \log N)$

```
ll f(int j, int i); // j->i로 전이 값 (dp[j] + cost(j, i))
void solve() {
  auto cross = [&](ll p, ll q) {
    ll lo = max(p, q), hi = n + 1;
    while (lo + 1 < hi) {
      ll mid = (lo + hi) / 2;
      if (f(p, mid) > f(q, mid)) hi = mid; // min 기준: f(p) >
      f(q)면 q가 우세
      else lo = mid;
    }
    return hi;
  };
  deque<pair<ll,ll>> dq; // {cand_idx, start_pos}
  dq.push_back({0, 1}); // 초기값: 0이 1번 위치부터 최적가정
  for (int i = 1; i <= n; i++) {
    while (dq.size() > 1 && dq[1].second <= i) dq.pop_front();
    dp[i] = f(dq[0].first, i);
    while (!dq.empty()) {
      ll p = dq.back().first, pos = cross(p, i);
      if (pos <= dq.back().second) dq.pop_back();
      else {
        if (pos <= n) dq.push_back({i, pos});
        break;
      }
    }
    if (dq.empty()) dq.push_back({i, 1});
  }
}
```

5.7 Aliens Trick

Usage: dp[t][i] = min(dp[t-1][j] + c[j+1][i]), c is Monge, find lambda w/ half bs. 정확히 K개 선택·분할하는 제약을 개수당 패널티로 대체
Time Complexity: $O(T \log X)$

```
/* n: 원소 개수 (경로 복원 끝점), k: 골라야 하는 개수
 * lo, hi: 패널티 이분탐색 범위 (0 ~ 최대 가치)
 * f(c): 패널티가 c일 때 {2*(가치합), prv} 반환 (가치는 c를 뺀
 값) */
template<class T, bool GET_MAX = false>
pair<T, vector<int>> AliensTrick(int n, int k, auto f, T lo,
T hi) {
  T l = lo, r = hi;
  while (l < r) {
    T m = (l + r + (GET_MAX ? 1 : 0)) >> 1;
    vector<int> prv = f(m * 2 + (GET_MAX ? -1 : 1)).second;
    int cnt = 0; for (int i = n; i; i = prv[i]) cnt++;
    if (cnt <= k) (GET_MAX ? l : r) = m;
    else (GET_MAX ? r : l) = m + (GET_MAX ? -1 : 1);
  }
  T opt_val = f(l * 2).first / 2 - k * l;
  auto get_path = [&](T c) {
    vector<int> p{n};
    for (auto prv = f(c).second; p.back(); )
    p.push_back(prv[p.back()]);
    reverse(p.begin(), p.end()); return p;
  };
  auto p1 = get_path(l * 2 + (GET_MAX ? 1 : -1));
  auto p2 = get_path(l * 2 - (GET_MAX ? 1 : -1));
  if (p1.size() == k + 1) return {opt_val, p1};
  if (p2.size() == k + 1) return {opt_val, p2};
  for (int i = 1, j = 1; i < p1.size(); i++) {
    while (j < p2.size() && p2[j] < p1[i - 1]) j++;
    if (p1[i] <= p2[j] && i - j == k + 1 - (int)p2.size()) {
      vector<int> res(p1.begin(), p1.begin() + i);
      res.insert(res.end(), p2.begin() + j, p2.end());
      return {opt_val, res};
    }
  }
  return {opt_val, {}}; // Should not reach here
}
```

5.8 Sum Over Subsets

Usage: dp[mask] = sum(A[i]), i is in mask
Time Complexity: $O(N2^N)$

```
// 1. Subset Sum: f[mask] = sum of a[sub] where sub & mask ==
sub
for (int i = 0; i < n; i++)
    for (int mask = 0; mask < (1 << n); mask++)
        if (mask & (1 << i)) f[mask] += f[mask ^ (1 << i)];
// 2. Superset Sum: f[mask] = sum of a[sup] where sup & mask
== mask
for (int i = 0; i < n; i++)
    for (int mask = (1 << n) - 1; mask >= 0; mask--)
        if (!(mask & (1 << i))) f[mask] += f[mask ^ (1 << i)];
```

5.9 Berlekamp Massey

Usage: 초기 항들에서 선형 점화식을 찾아 N번째 항 계산
Time Complexity: N-th term: $O(K^2 \log N)$ / Det: $O(N(N+E))$

```
int randint(int lb, int ub) { return
uniform_int_distribution<int>(lb, ub)(rng); }
const int mod = 998244353;
ll ipow(ll x, ll p) {
  ll ret = 1, piv = x;
  while (p) {
    if (p & 1) ret = ret * piv % mod;
    piv = piv * piv % mod; p >>= 1;
  }
  return ret;
}
vector<int> berlekamp_massey(vector<int> x) {
  vector<int> ls, cur; int lf, ld;
  for (int i = 0; i < sz(x); i++) {
    ll t = 0;
    for (int j = 0; j < sz(cur); j++) t = (t +
    1ll*x[i-j-1]*cur[j]) % mod;
    if ((t - x[i]) % mod == 0) continue;
    if (cur.empty()) {
      lf = i; ld = (t-x[i]) % mod;
      cur.resize(i+1); continue;
    }
    ll k = -(x[i]-t) * ipow(ld, mod-2) % mod;
    vector<int> c(i-lf-1); c.push_back(k);
    for (auto& j : ls) c.push_back(-j*k % mod);
    if (sz(c) < sz(cur)) c.resize(sz(cur));
    for (int j = 0; j < sz(cur); j++) c[j] = (c[j]+cur[j]) %
    mod;
    if (i-lf+sz(ls) >= sz(cur)) {
      tie(ls, lf, ld) = make_tuple(cur, i, (t-x[i]) % mod);
    } cur = c;
  }
  for (auto& i : cur) i = (i % mod + mod) % mod;
  return cur;
}
int get_nth(vector<int> rec, vector<int> dp, ll n) {
  int m = sz(rec); vector<int> s(m), t(m); s[0] = 1;
  if(m != 1) t[1] = 1; else t[0] = rec[0];
  auto mul = [&rec](vector<int> v, vector<int> w) {
    int m = sz(v); vector<int> t(2*m);
    for (int j = 0; j < m; j++) {
      for (int k = 0; k < m; k++) {
        t[j+k] += 1ll*v[j]*w[k] % mod;
        if (t[j+k] >= mod) t[j+k] -= mod;
      }
    }
    for (int j = 2*m-1; j >= m; j--) {
      for (int k = 1; k <= m; k++) {
        t[j-k] += 1ll*t[j]*rec[k-1] % mod;
        if (t[j-k] >= mod) t[j-k] -= mod;
      }
    }
    t.resize(m); return t;
  };
  while (n) {
    if (n & 1) s = mul(s, t);
    t = mul(t, t); n >>= 1;
  }
  ll ret = 0;
  for (int i = 0; i < m; i++) ret += 1ll*s[i]*dp[i] % mod;
  return ret % mod;
}
int guess_nth_term(vector<int> x, ll n) {
  if (n < sz(x)) return x[n];
  vector<int> v = berlekamp_massey(x);
  if (v.empty()) return 0;
  return get_nth(v, x, n);
}
struct elem { int x, y, v; }; // A_(x, y) <- v, 0-based. no
duplicate
vector<int> get_min_poly(int n, vector<elem> M) {
  // smallest poly P such that A^i = sum_{j < i} {A^j \times
  P_j}
  vector<int> rnd1, rnd2;
  for (int i = 0; i < n; i++) {
    rnd1.push_back(randint(1, mod - 1));
    rnd2.push_back(randint(1, mod - 1));
  }
  vector<int> gobs;
  for (int i = 0; i < 2*n+2; i++) {
    int tmp = 0;
```

```
    for (int j = 0; j < n; j++) {
      tmp += 1ll * rnd2[j] * rnd1[j] % mod;
      if (tmp >= mod) tmp -= mod;
    }
    gobs.push_back(tmp); vector<int> nxt(n);
    for (auto& i : M) {
      nxt[i.x] += 1ll * i.v * rnd1[i.y] % mod;
      if(nxt[i.x] >= mod) nxt[i.x] -= mod;
    }
    rnd1 = nxt;
  }
  auto sol = berlekamp_massey(gobs);
  reverse(all(sol)); return sol;
}
ll det(int n, vector<elem> M) {
  vector<int> rnd;
  for (int i = 0; i < n; i++) rnd.push_back(randint(1,
  mod-1));
  for (auto& i : M) i.v = 1ll * i.v * rnd[i.y] % mod;
  auto sol = get_min_poly(n, M)[0]; if (n%2 == 0) sol =
  mod-sol;
  for (auto& i : rnd) sol = 1ll * sol * ipow(i, mod-2) % mod;
  return sol;
}
```

6 Geometry

6.1 Geometry Template

```
const double EPS = 1e-9;
template<typename T> struct Point {
  T x, y;
  bool operator<(const Point& p) const { return x==p.x ?
  y<p.y : x<p.x; }
  bool operator==(const Point& p) const { return x==p.x &&
  y==p.y; }
  Point operator+(const Point& p) const { return {x+p.x,
  y+p.y}; }
  Point operator-(const Point& p) const { return {x-p.x,
  y-p.y}; }
  Point operator*(T n) const { return {x*n, y*n}; }
  Point operator/(T n) const { return {x/n, y/n}; }
  T operator*(const Point& p) const { return x*p.x + y*p.y; }
  T operator/(const Point& p) const { retur n x*p.y - y*p.x; }
  /* 3D dot, cross
  T operator*(const Point& p) const { return x*p.x + y*p.y +
  z*p.z; }
  Point operator/(const Point& p) const {
    return {y*p.z - z*p.y, z*p.x - x*p.z, x*p.y - y*p.x};
  } */
  T dist2() const { return x*x + y*y; }
  Point<double> d() const { return {(double)x, (double)y}; }
  Point rot90() const { return {-y, x}; }
};
using P = Point<ll>;
using Pd = Point<double>;
int ccw(P a, P b, P c) {
  ll res = (b - a) / (c - a);
  return (res > 0) - (res < 0);
}
bool isInter(P a, P b, P c, P d) {
  int ab = ccw(a, b, c) * ccw(a, b, d), cd = ccw(c, d, a) *
  ccw(c, d, b);
  if (ab == 0 && cd == 0) {
    if (b < a) swap(a, b); if (d < c) swap(c, d);
    return !(b < c || d < a);
  }
  return ab <= 0 && cd <= 0;
}
ll area2(const vector<P>& v) { // 다각형 넓이*2
  ll res = 0; int n = sz(v);
  for (int i = 0; i < n; i++) res += v[i] / v[(i+1)%n];
  return abs(res);
}
```

6.2 Closest Pair

Time Complexity: $O(N \log N)$

```
struct P { ll x, y; int id; };
ll dist(const P &a, const P &b) {
  return (a.x-b.x)*(a.x-b.x) + (a.y-b.y)*(a.y-b.y);
}
array<ll,3> closestPair(vector<P> &v) {
  sort(all(v), [](const P &a, const P &b) { return a.x <
  b.x; });
  auto cmp = [](const P &a, const P &b) {
    return a.y == b.y ? a.x < b.x : a.y < b.y;
  };
  set<P,decltype(cmp)> s(cmp);
  ll d = 9e18; array<ll,3> ans = { d, -1, -1 };
  for (int i = 0, j = 0; i < sz(v); i++) {
    while (j < i && (v[i].x-v[j].x)*(v[i].x-v[j].x) >= d)
    s.erase(v[j++]);
    ll dsq = sqrt(d)+1;
    auto it1 = s.lower_bound({ LLONG_MIN, v[i].y - dsq, -1 });
    auto it2 = s.upper_bound({ LLONG_MAX, v[i].y + dsq, -1 });
    for (auto it = it1; it != it2; it++) {
      ll cur = dist(v[i], *it);
      if (cur < d) ans = { d = cur, v[i].id, (*it).id };
    }
    s.insert(v[i]);
  }
  return ans;
}
```

6.3 Convex Hull

Usage: Returns vertices in CCW order.
Time Complexity: $O(N \log N)$

```
vector<P> ConvexHull(vector<P> ps) { // Monotone chain
  compress(ps); if (sz(ps) <= 2) return ps;
  vector<P> v(sz(ps)*2);
  int s = 0, t = 0;
  for (int i = 2; i--; s = --t, reverse(all(ps))) {
    for (P p : ps) { // include boundary : ccw < 0
      while (t >= s+2 && ccw(v[t-2], v[t-1], p) <= 0) t--;
      v[t++] = p;
    }
  }
  v.resize(t + (t <= 1)); return v;
}
```

6.4 Convex Hull 3d

Time Complexity: $O(N^2)$

```
struct face { int a, b, c; P d; };
vector<face> ConvexHull_3d(vector<P> &p) {
  compress(p); int n = sz(p);
  if (n <= 3) return {};
  int k = 2;
  while (k < n && ((p[1]-p[0]) / (p[k]-p[0])) == P{0, 0, 0})
  k++;
  if (k == n) return {};
  swap(p[2], p[k]);
  k = 3;
  while (k < n && (((p[1]-p[0]) / (p[2]-p[0])) *
  (p[k]-p[0])) == 0) k++;
  if (k == n) return {};
  swap(p[3], p[k]);
  vector<face> f;
  vector<vector<bool>> dead(n, vector<bool>(n, true));
  auto add_face = [&](int a, int b, int c) {
    f.push_back({ a, b, c, (p[b]-p[a]) / (p[c]-p[a]) });
    dead[a][b] = dead[b][c] = dead[c][a] = 0;
  };
  add_face(0, 1, 2); add_face(0, 2, 1);
  for (int i = 3; i < n; i++) {
    vector<face> nf;
    for (face &F : f){
      if ((p[i] - p[F.a]) * F.d > 0) {
        dead[F.a][F.b] = dead[F.b][F.c] = dead[F.c][F.a] = 1;
      } else {
        nf.push_back(F);
      }
    }
    f = nf;
    for (face &F : nf){
      if (dead[F.b][F.a]) add_face(F.b, F.a, i);
      if (dead[F.c][F.b]) add_face(F.c, F.b, i);
      if (dead[F.a][F.c]) add_face(F.a, F.c, i);
    }
  }
  return f;
}
```

6.5 Rotating Calipers

Usage: Calculates the diameter (farthest pair of points) of a convex hull(CCW order).
Time Complexity: $O(N)$

```
// ccw 정렬된 convex에서 가장 먼 두 점
pair<P,P> Calipers(const vector<P>& v) {
  int n = sz(v); if (n < 2) return {v[0], v[0]};
  ll mx = 0; P a = v[0], b = v[1];
  for (int i = 0, j = 1; i < n; i++) {
    P t = v[(i+1)%n] - v[i];
    while ((t / (v[(j+1)%n] - v[j])) > 0) j = (j+1)%n;
    ll now = (v[i] - v[j]).dist2();
    if (now > mx) mx = now, a = v[i], b = v[j];
  }
  return { a, b };
}
```

6.6 Point in Polygon

Time Complexity: $O(\log N)$, $O(N)$

```
// 다각형 내부 판별 (CCW/CW 순서 무관), O(N)
bool InPoly(const vector<P>& v, P p) {
  int n = sz(v); bool chk = false;
  for (int i = 0; i < n; i++) {
    P a = v[i], b = v[(i+1)%n];
    ll cp = (b-a)/(p-a);
    if (!cp && (a-p)*(b-p) <= 0) return true; // false:
    exclude boundary
    if ((a.y > p.y) != (b.y > p.y) && (cp > 0) == (b.y > a.y))
      chk = !chk;
  }
  return chk;
}
// CCW 정렬된 다각형 내부 판별, O(log N)
bool InConvPoly(const vector<P>& v, P p) {
  int n = sz(v); if (n<3) return false;
  // ccw <= 0 || ccw >= 0: exclude boundary
  if (ccw(v[0], v[1], p) < 0 || ccw(v[0], v.back(), p) > 0)
  return false;
  int l = 1, r = n - 1;
  while (l + 1 < r) {
    int mid = (l + r) / 2;
    if (ccw(v[0], v[mid], p) >= 0) l = mid;
    else r = mid;
  }
  return ccw(v[l], v[l + 1], p) >= 0; // > 0: exclude boundary
}
```

6.7 Angular sort

Usage: Angular sort. 1. Relative to pivot (Convex Hull). 2. Relative to origin.

```
// cent 기준 [0,360) 각도순. 같은 방향은 가까운 점부터.
// cent와 같은 점은 맨 앞.
void Sort(vector<P>& v, P cent) {
  auto half = [](P p) { return p.y < 0 || (p.y == 0 && p.x <
  0); };
  sort(all(v), [&](P a, P b) {
    a = a-cent; b = b-cent;
    if (half(a) != half(b)) return half(a) < half(b);
    ll cp = a/b;
    return cp ? cp > 0 : a.dist2() < b.dist2();
  });
}
// 최하단·최좌측 점 기준 각도순.
void Sort(vector<P>& v) {
  if (v.empty()) return;
  P cent = *min_element(all(v), [](P a, P b) {
    return a.y != b.y ? a.y < b.y : a.x < b.x;
  });
  Sort(v, cent);
}
// Sort(v); / Sort(v, {0,0});
```

6.8 Linear Minkowski Sum

Usage: Minkowski Sum of Two convex(must be CCW order).

Time Complexity: $O(N+M)$

```
vector<P> Minkowski(vector<P> p, vector<P> q) {
  if (p.empty() || q.empty()) return {};
  rotate(p.begin(), min_element(all(p)), p.end());
  rotate(q.begin(), min_element(all(q)), q.end());
  p.push_back(p[0]); p.push_back(p[1]);
  q.push_back(q[0]); q.push_back(q[1]);
  vector<P> res; int i = 0, j = 0;
  while (i < sz(p) - 2 || j < sz(q) - 2) {
    res.push_back(p[i] + q[j]);
    ll cp = (p[i + 1] - p[i]) / (q[j + 1] - q[j]);
    if (cp >= 0 && i < sz(p) - 2) i++;
    if (cp <= 0 && j < sz(q) - 2) j++;
  }
  return res;
}
```

6.9 Smallest Enclosing Circle

Usage: Welzl's algorithm to find the Minimum Enclosing Circle. Returns center, radius.

Time Complexity: Expected $O(N)$

```
Pd getC(Pd a, Pd b) { return (a + b) / 2.0; }
Pd getC(Pd a, Pd b, Pd c) {
  Pd u = b-a, v = c-a; double d = 2*(u/v);
  if (abs(d) < EPS) return {0, 0};
  return a - (v * u.dist2() - u * v.dist2()).rot90() / d;
}
/* // for 3D
Pd getC(Pd a, Pd b, Pd c) {
  Pd u = b-a, v = c-a, n = u/v;
  if (n.dist2() < EPS) return getC(a, b);
  Pd top = (n/u) * v.dist2() + (v/n) * u.dist2();
  return a + top / (2*n.dist2());
}
Pd getC(Pd a, Pd b, Pd c, Pd d) {
  Pd p = b-a, q = c-a, r = d-a;
  double det = ((p / q) * r) * 2.0;
  if (abs(det) < EPS) return a;
  Pd top = (p/q) * r.dist2() + (q/r) * p.dist2() + (r/p) *
  q.dist2();
  return a + top / det;
} */
pair<Pd,double> min_circle(vector<Pd> v) {
  shuffle(all(v), rng);
  Pd c = { 0, 0 }; double r = 0;
  auto dist = [&](Pd p1, Pd p2) { return
  sqrt((p1-p2).dist2()); };
  auto chk = [&](Pd p) { return dist(c, p) > r+EPS; };
  for (int i = 0; i < sz(v); i++) if (chk(v[i])) {
    c = v[i]; r = 0;
    for (int j = 0; j < i; j++) if (chk(v[j])) {
      c = getC(v[i], v[j]); r = dist(c, v[i]);
      for (int k = 0; k < j; k++) if (chk(v[k])) {
        c = getC(v[i], v[j], v[k]); r = dist(c, v[k]);
        /* // for 3D
        for (int l = 0; l < k; l++) if (chk(v[l])) {
          c = getC(v[i], v[j], v[k], v[l]); r = dist(c, v[l]);
        } */
      }
    }
  }
  return { c, r };
}
```

6.10 Geometric Intersections

Usage: Intersection primitives: Segment, Line-Circle, Line-Hull, and Circle-Polygon area.

Time Complexity: $O(1)/O(1)/O(\log N)/O(N)$

```
using T = __int128_t; // T <= O(COORD^3)
// [1] 선분 교차 (정밀 좌표) / Param: 선분 ab, 선분 cd
// Return: {flag, xn, xd, yn, yd} -> 교점 (xn/xd, yn/yd)
// Flag: 0(안만남), 1(교점=끝점), 4(교차), -1(무수히 겹침)
tuple<int, T, T, T, T> segmentInter(P a, P b, P c, P d) {
  if (!isInter(a, b, c, d)) return {0, 0, 0, 0, 0};
  T det = (b - a) / (d - c);
  if (det == 0) {
    if (b < a) swap(a, b);
    if (d < c) swap(c, d);
    if (b == c) return {1, b.x, 1, b.y, 1};
    if (d == a) return {1, d.x, 1, d.y, 1};
    return {-1, 0, 0, 0, 0};
  }
  T p = (c - a) / (d - c), q = det;
  T xp = a.x * q + (b.x - a.x) * p, xq = q;
  T yp = a.y * q + (b.y - a.y) * p, yq = q;
  if (xq < 0) { xp = -xp; xq = -xq; }
  if (yq < 0) { yp = -yp; yq = -yq; }
  T g_x = __gcd(xp, xq); xp /= g_x; xq /= g_x;
  T g_y = __gcd(yp, yq); yp /= g_y; yq /= g_y;
  int f = 4;
  if ((xp == a.x * xq && yp == a.y * yq) || (xp == b.x * xq
  && yp == b.y * yq)) f = 1;
  if ((xp == c.x * xq && yp == c.y * yq) || (xp == d.x * xq
  && yp == d.y * yq)) f = 1;
  return { f, xp, xq, yp, yq };
}
// [2] 원-직선 교차 / Param: 직선 ab, 원(c, r)
// Return: 교점 좌표 목록 (a -> b 방향 순서 정렬됨)
vector<Pd> lineCircle(P a, P b, P c, double r) {
  P ab = b - a;
  Pd p = a.d() + ab.d() * ((c - a) * ab / (double)ab.dist2());
  double h2 = r * r - (p - c.d()).dist2();
  if (h2 < -EPS) return {};
  if (abs(h2) < EPS) return {p};
  Pd h = ab.d() * (sqrt(h2) / sqrt(ab.dist2()));
  return {p - h, p + h};
}
// [3] 원-다각형 교차 넓이
// Param: 원(c, r), 다각형 poly (CCW/CW 무관) Return: 교차하는
영역의 넓이
double areaCirclePoly(P c, double r, const vector<P>& poly) {
  auto tri = [&](Pd p, Pd q) {
    Pd d = q - p;
    double a = d * p / d.dist2(), b = (p.dist2() - r * r) /
    d.dist2();
    double det = a * a - b;
    if (det <= EPS) return r * r * atan2(p / q, p * q);
    double t1 = -a-sqrt(max(0.0, det)), t2 =
    -a+sqrt(max(0.0, det));
```

```
    if (t2 < -EPS || t1 > 1.0 + EPS) return r * r * atan2(p
    / q, p * q);
    Pd u = p + d * max(0.0, t1), v = p + d * min(1.0, t2);
    return r * r * atan2(p / u, p * u) + u / v + r * r *
    atan2(v / q, v * q);
  };
  double res = 0; int n = poly.size();
  for(int i = 0; i < n; i++)
    res += tri((poly[i] - c).d(), (poly[(i+1) % n] - c).d());
  return res * 0.5;
}
// [4] 볼록 다각형-직선 교차 (O(log N)) Param: 볼록 다각형 h
(CCW), 직선 ab
// Return: {i, j} -> 직선이 변(i, i+1)과 변(j, j+1)을 교차함.
안만나면 {-1, -1}
int extrVertex(const vector<P>& h, P dir) {
  int n = h.size();
  auto cmp = [&](int i, int j) { return ccw({0,0}, dir,
  h[i%n] - h[j%n]); };
  auto isExtr = [&](int i) { return cmp(i, i - 1 + n) >= 0
  && cmp(i, i + 1) > 0; };
  if (isExtr(0)) return 0;
  int l = 0, r = n;
  while (l + 1 < r) {
    int m = (l + r) / 2;
    if (isExtr(m)) return m;
    if (cmp(l + 1, l) > 0) {
      if (cmp(m + 1, m) > 0 && cmp(l, m) > 0) r = m; else l
      = m;
    } else {
      if (cmp(m + 1, m) <= 0 && cmp(l, m) < 0) l = m; else r
      = m;
    }
  }
  return l;
}
array<int,2> hullLineInter(const vector<P>& h, P a, P b) {
  P dir = b - a, per = {-dir.y, dir.x};
  int n = h.size();
  int i1 = extrVertex(h, per), i2 = extrVertex(h, per * -1);
  if (ccw(a, b, h[i1]) * ccw(a, b, h[i2]) > 0) return {-1,
  -1};
  auto search = [&](int s, int e) {
    if (ccw(a, b, h[s]) == 0) return s;
    int l = s, r = e;
    if (r < l) r += n;
    while (l + 1 < r) {
      int m = (l + r) / 2;
      if (ccw(a, b, h[m % n]) == ccw(a, b, h[s])) l = m;
      else r = m;
    }
    return l % n;
  };
  return {search(i1, i2), search(i2, i1)};
}
```

6.11 Half Plane Intersection

Usage: 일차 부등식의 공통 영역·볼록 다각형 교집합·다각형 kernel
Time Complexity: $O(N \log N)$

```
// Line(p1,p2): p1 -> p2의 오른쪽 반평면, 경계 포함. p1 != p2.
// 유효 영역: a*x + b*y <= c.
// CCW 다각형 내부를 취하려면 Line(다음 정점, 현재 정점).
struct Line {
  double a, b, c; // ax + by <= c
  Line(Pd p1, Pd p2) {
    a = p1.y - p2.y; // -dy
    b = p2.x - p1.x; // dx
    c = a * p1.x + b * p1.y;
  }
  Pd slope() const { return {a, b}; }
};
Pd intersect(Line u, Line v) { // 평행하지 않은 두 직선의 교점
  double det = u.a * v.b - u.b * v.a;
  return {(u.c * v.b - u.b * v.c) / det, (u.a * v.c - u.c *
  v.a) / det};
}
bool bad(Line l, Pd p) {
  return l.a * p.x + l.b * p.y > l.c + EPS;
}
// 유계·양의 면적 교집합의 꼭짓점을 구함. O(N log N).
// 빈 영역/점/선분/무계 영역의 판별은 보장하지 않음.
// EPS 정렬 조건: 비평행 법선 간 외적의 절댓값이 EPS보다 커야
함.
vector<Pd> HPI(vector<Line> lines) {
  sort(all(lines), [&](const Line& u, const Line& v) {
    Pd p1 = u.slope(), p2 = v.slope();
    bool f1 = p1.y > 0 || (p1.y == 0 && p1.x > 0);
    bool f2 = p2.y > 0 || (p2.y == 0 && p2.x > 0);
    if (f1 != f2) return f1 > f2;
    if (abs(p1 / p2) > EPS) return (p1 / p2) > 0;
    return u.c < v.c; // 같은 방향 평행선은 법선 길이가 같아야
    c 비교가 유효
  });
  deque<Line> dq;
  for (auto& l : lines) {
    if (!dq.empty() && abs(dq.back().slope() / l.slope()) <
    EPS) continue;
    while (sz(dq) >= 2 && bad(l, intersect(dq.back(),
    dq[sz(dq)-2]))) dq.pop_back();
    while (sz(dq) >= 2 && bad(l, intersect(dq[0], dq[1])))
    dq.pop_front();
    dq.push_back(l);
  }
  while (sz(dq) > 2 && bad(dq[0], intersect(dq.back(),
  dq[sz(dq)-2]))) dq.pop_back();
  while (sz(dq) > 2 && bad(dq.back(), intersect(dq[0],
  dq[1]))) dq.pop_front();
  vector<Pd> res; if (sz(dq) < 3) return {};
  for (int i = 0; i < sz(dq); i++) {
    res.push_back(intersect(dq[i], dq[(i + 1) % sz(dq)]));
  }
  return res;
}
```

6.12 Manhattan MST

Usage: 맨해튼 거리의 완전 그래프 MST
Time Complexity: $O(N \log N)$

```
vector<array<ll,3>> manhattan_mst(vector<P> v) {
  vector<int> idx(sz(v));
  for (int i = 0; i < sz(idx); i++) idx[i] = i;
  vector<array<ll,3>> edges;
  for (int rot = 0; rot < 4; rot++) {
    sort(all(idx), [&v](int i, int j){ return
    (v[i].x+v[i].y) < (v[j].x+v[j].y); });
    map<int,int,greater<int>> active;
    for (auto i : idx) {
      for (auto it = active.lower_bound(v[i].x); it !=
      active.end(); active.erase(it++)) {
        int j = it->second;
        if (v[i].x-v[i].y > v[j].x-v[j].y) break;
        edges.push_back({(v[i].x-v[j].x) + (v[i].y-v[j].y),
        i+1, j+1});
      }
      active[v[i].x] = i;
    }
    for (auto &p : v) {
      if (rot & 1) p.x *= -1;
      else swap(p.x, p.y);
    }
  }
  sort(all(edges));
  return edges;
}
```

7 String

7.1 Aho-Corasick

Usage: Multi-pattern matching
Time Complexity: $O(\sum |P| + |T|)$

```
// 소문자 패턴. insert 전부 -> build 한 번 -> find/count.
// find: 서로 다른 패턴, id>=1. 반환 {패턴id, 끝idx}.
// count: 겹치는 등장도 모두 셈. 같은 패턴의 중복 삽입도
횟수에 반영.
const int CH = 26;
struct AhoCorasick {
  struct Node {
    int ch[CH]{}, id = 0, fail = 0, out = 0, cnt = 0;
  };
  vector<Node> T; int tc = 1; // 루트=1, 0은 없는 간선
  AhoCorasick(int SZ) : T(SZ) {}
  int ID(char c) { return c-'a'; }
  void insert(const string& s, int id) {
    int x = 1;
    for (char c : s) {
      int v = ID(c);
      if (!T[x].ch[v]) T[x].ch[v] = ++tc;
      x = T[x].ch[v];
    }
    T[x].id = id; T[x].cnt++;
  }
  void build() {
    queue<int> q;
    for (int i = 0; i < CH; i++) {
      int& u = T[1].ch[i];
```

```
      if (u) T[u].fail = 1, q.push(u);
      else u = 1;
    }
    while (!q.empty()) {
      int v = q.front(); q.pop();
      int f = T[v].fail;
      T[v].cnt += T[f].cnt;
      T[v].out = T[f].id ? f : T[f].out;
      for (int i = 0; i < CH; i++) {
        int& u = T[v].ch[i];
        if (u) T[u].fail = T[f].ch[i], q.push(u);
        else u = T[f].ch[i];
      }
    }
  }
  int next(int x, int c) { return T[x].ch[c]; }
  vector<pair<int,int>> find(const string& s) {
    vector<pair<int,int>> res; int x = 1;
    for (int i = 0; i < sz(s); i++) {
      x = next(x, ID(s[i]));
      if (T[x].id) res.emplace_back(T[x].id, i);
      for (int y = T[x].out; y; y = T[y].out)
        res.emplace_back(T[y].id, i);
    }
    return res;
  }
  ll count(const string& s) {
    ll res = 0; int x = 1;
    for (char c : s) res += T[x = next(x, ID(c))].cnt;
    return res;
  }
};
// AhoCorasick ac(패턴 길이 합 + 2);
// ac.insert(p, id); -> ac.build();
// ac.find(s); // {id, 끝idx} 목록
// ac.count(s); // 총 등장 횟수
```

7.2 Rolling Hash

Time Complexity: $O(N)$

```
template<ll P, ll M> struct Hash1 {
  vector<ll> h, p;
  void build(const string& s) {
    int n = sz(s); h.assign(n+1, 0); p.assign(n+1, 1);
    for (int i = 0; i < n; i++) {
      h[i+1] = (h[i]*P + s[i])%M;
      p[i+1] = p[i]*P%M;
    }
  }
  ll get(int l, int r) const { // [l,r)
    return (h[r]-h[l]*p[r-l]%M+M)%M;
  }
};
template<ll P1, ll M1, ll P2, ll M2> struct Hashing {
  Hash1<P1,M1> a; Hash1<P2,M2> b;
  void build(const string& s) { a.build(s); b.build(s); }
  pair<ll,ll> get(int l, int r) const {
    return { a.get(l,r), b.get(l,r) };
  }
};
// 1e5+3, 1e5+13, 131'071, 524'287, 1'299'709, 1'301'021
// 1e9-63, 1e9+7, 1e9+9, 1e9+103
// using Hash = Hashing<917, 998244353, 10009, 1000000007>;
// Hash H; H.build(s);
// H.get(l,r) == H.get(l2,r2): s[l..r-1]과 s[l2..r2-1] 비교
```

7.3 KMP

Usage: Single pattern matching

Time Complexity: $O(N+M)$

```
// minPeriod(): 최소 주기. s = p^k 꼴이면 |p|, 아니면 len(s)
// pi[i]: prefix [0..i]의 최장 접두사=접미사 길이
template <typename T> struct KMP {
  T P; vector<int> pi;
  KMP(const T& P) : P(P), pi(sz(P)) {
    for (int i = 1, j = 0; i < sz(P); i++) {
      while (j > 0 && P[i] != P[j]) j = pi[j-1];
      if (P[i] == P[j]) pi[i] = ++j;
    }
  }
  vector<int> find(const T& S) {
    vector<int> res;
    int n = sz(S), m = sz(P), j = 0;
    for (int i = 0; i < n; i++) {
      while (j > 0 && S[i] != P[j]) j = pi[j-1];
      if (S[i] == P[j]) {
        if (j == m-1)
          res.push_back(i-m+1), j = pi[j];
        else j++;
      }
    }
    return res;
  }
  int minPeriod() {
    int m = sz(P); if (m == 0) return 0;
    int len = m-pi[m-1]; if (m%len == 0) return len;
    return m;
  }
};
```

7.4 Manacher

Usage: 중심별 회문 반지름·최장 회문·회문 개수·구간 회문 판정

Time Complexity: $O(N)$

```
// 각 문자를 중심으로 하는 최장 팰린드롬의 반경을 반환
// Manacher("abaaba") = {0,1,0,3,0,1,6,1,0,3,0,1,0}
// # a # b # a # a # b # a #
// 0 1 0 3 0 1 6 1 0 3 0 1 0
struct Manacher {
  vector<int> p;
  template<class T> Manacher(const T &s) {
    int n = sz(s)*2+1, c = 0, r = 0;
    p.assign(n, 0); T t; t.resize(n);
    for (int i = 0; i < sz(s); i++) t[i*2+1] = s[i];
    for (int i = 0; i < n; i++) {
      if (r > i) p[i] = min(r-i, p[2*c-i]);
      while (i-p[i]-1 >= 0 && i+p[i]+1 < n && t[i-p[i]-1] ==
      t[i+p[i]+1]) p[i]++;
      if (i+p[i] > r) c = i, r = i+p[i];
    }
  }
  bool chk(int l, int r) {
    if (l > r) return false;
    return p[l+r+1] >= r-l+1;
  }
  ll cnt() {
    ll res = 0;
    for (auto i : p) res += (i+1)/2;
    return res;
  }
};
```

7.5 Suffix Array

Usage: 부분문자열의 사전순·반복 부분문자열·suffix 간 LCP

Time Complexity: $sa : O(N \log N), lcp : O(N)$

```
// Suffix Array – 모든 접미사 사전순 정렬 + LCP
//  sa[i] : i번째 사전순 작은 접미사의 시작 인덱스 (0-idx)
//  lcp[i] : sa[i-1] ~ sa[i] 공통 접두사 길이 (lcp[0]=0)
struct SA {
  int n;
  vector<int> sa, lcp, x, y, c;
  SA(const string& s) : n(sz(s)), sa(n), lcp(n), x(2*n, -1),
  y(2*n, -1), c(max(n, 256)) {
    for (int i = 0; i < n; i++) c[x[i] = s[i]]++;
    for (int i = 1; i < sz(c); i++) c[i] += c[i-1];
    for (int i = n - 1; i >= 0; i--) sa[--c[x[i]]] = i;
    for (int d = 1, p = 0; ; d <<= 1, c.resize(p+1), p = 0) {
      for (int i = n - d; i < n; i++) y[p++] = i;
      for (int i = 0; i < n; i++) if (sa[i] >= d) y[p++] =
      sa[i] - d;
      fill(all(c), 0);
      for (int i = 0; i < n; i++) c[x[y[i]]]++;
      for (int i = 1; i < sz(c); i++) c[i] += c[i-1];
      for (int i = n - 1; i >= 0; i--) sa[--c[x[y[i]]]] =
      y[i];
      swap(x, y); p = x[sa[0]] = 0;
      for (int i = 1; i < n; i++)
      x[sa[i]] = (y[sa[i-1]] == y[sa[i]] && y[sa[i-1]+d] ==
      y[sa[i]+d]) ? p : ++p;
      if (p == n-1) break;
    }
    for (int i = 0, k = 0; i < n; i++, k = max(0, k-1)) {
      if (x[i] == 0) continue;
      for (int j = sa[x[i]-1]; s[i+k] == s[j+k]; k++);
      lcp[x[i]] = k;
    }
  }
};
```

7.6 Z-algorithm

Usage: 각 위치와 접두사의 일치 길이·접두사 등장 횟수·반복 구조

Time Complexity: $O(N)$

```
// Z[i] = LongestCommonPrefix(S[0:N], S[i:N])
//    = S[0:N]과 S[i:N]이 앞에서부터 몇 글자 겹치는지
vector<int> Z(const string &s){
  int n = s.size();
```

```
  vector<int> z(n);
  z[0] = n;
  for(int i=1, l=0, r=0; i<n; i++){
    if(i < r) z[i] = min(r-i-1, z[i-l]);
    while(i+z[i] < n && s[i+z[i]] == s[z[i]]) z[i]++;
    if(i+z[i] > r) r = i+z[i], l = i;
  }
  return z;
}
```

7.7 Eertree

Usage: 서로 다른 회문을 길이·등장 횟수별로 관리. 중심별 반지름만 필요하면 Manacher
Time Complexity: $O(N \log \Sigma)$

```
// Eertree(palindromic tree): "서로 다른 팰린드롬 부분
문자열"을 노드로 압축.
// 각 노드 = 팰린드롬 하나. 간선 = 양끝에 같은 문자 붙여 만든
더 긴 팰린드롬.
// "distinct 팰린드롬" 개수·등장 횟수
//  for (char c : s) et.add(c); // 문자열 순서대로 삽입
(온라인)
//  et.count(); // 등장 횟수
//  서로 다른 팰린드롬 개수 = sz(tree) - 2 // 노드 0,1은
가상/빈 루트
//  tree[i].l  = 그 팰린드롬 길이
//  tree[i].c  = 등장 횟수 (count() 후에만 정확)
//  tree[i].f  = suffix link (가장 긴 진짜 접미사 팰린드롬)
struct EERTREE {
  struct Node {
    int l, f, c, p, nxt[26];
    Node(int l, int f, int p = 0) : l(l), f(f), c(0), p(p) {
      for (int i = 0; i < 26; i++) nxt[i] = 0;
    }
  };
  vector<Node> tree; string s; int last;
  void reset() { s = "#"; last = 1; }
  EERTREE() { reset();
    tree.push_back({ -1, 0 }); tree.push_back({ 0, 0 });
  }
  int find(int x) {
    int p = sz(s)-1;
    while (s[p - tree[x].l - 1] != s[p]) x = tree[x].f;
    return x;
  }
  void add(char c) {
    s += c; int v = c-'a', p = find(last);
    if (!tree[p].nxt[v]) {
      int lnk = (p == 0 ? 1 : tree[find(tree[p].f)].nxt[v]);
      tree.push_back({ tree[p].l+2, lnk, p });
      tree[p].nxt[v] = sz(tree)-1;
    }
    tree[last = tree[p].nxt[v]].c++;
  }
  void count() {
    for (int i = sz(tree)-1; i > 1; i--) tree[tree[i].f].c
    += tree[i].c;
  }
};
```

7.8 Suffix Automaton

Usage: 서로 다른 부분문자열의 개수·등장 횟수·최장 공통 부분문자열. 사전순·LCP 중심이면 Suffix Array
Time Complexity: $O(N \cdot \Sigma)$

```
template<typename T, size_t S, T init_val>
struct initialized_array : public array<T, S> {
  initialized_array() { this->fill(init_val); }
};

template<class Char_Type, class Adjacency_Type>
struct suffix_automaton {
  vector<int> len{0}, link{-1}, firstpos{-1}, is_clone{0};
  vector<Adjacency_Type> next{{}};
  ll ans = 0; int last = 0;
  int new_state(int l, int sl, int fp, bool c,
                const Adjacency_Type& adj) {
    int v = sz(len);
    len.push_back(l); link.push_back(sl);
    firstpos.push_back(fp);
    is_clone.push_back(c); next.push_back(adj); return v;
  }
  void extend(const vector<Char_Type>& s) {
    last = 0; for (auto c : s) extend(c);
  }
  void extend(Char_Type c) {
    int cur = new_state(len[last]+1, 0, len[last], false,
    {}), p = last;
    while (p != -1 && !next[p][c])
      next[p][c] = cur, p = link[p];
    if (p != -1) {
      int q = next[p][c];
      if (len[p]+1 == len[q]) link[cur] = q;
      else {
        int clone = new_state(len[p]+1, link[q],
        firstpos[q], true, next[q]);
        while (p != -1 && next[p][c] == q)
          next[p][c] = clone, p = link[p];
        link[cur] = link[q] = clone;
      }
    }
    last = cur; ans += len[cur]-len[link[cur]];
  }
  int size() const { return sz(len); }
};
// T.ans: 서로 다른 부분문자열 개수
// T.next[v][c]: 전이. 0이면 없음, 루트=0
// T.len[v]: 상태의 최대 길이, T.link[v]: suffix link
// T.firstpos[v]: 최대 길이 문자열의 최초 등장 끝idx
// 등장 횟수: 일반 상태 1, 클론 0으로 시작해 len 내림차순으로
link에 누적.
suffix_automaton<int, initialized_array<int,26,0>> T;
for (char c : s) T.extend(c-'a');
// 문자열 p가 s의 부분문자열인지 판정
int x = 0; for (char c : p) if ((x = T.next[x][c-'a']) == 0)
return false;
```

8 Policy-Based Data Structures(pbds)

8.1 Hash map (pb_ds)

Time Complexity: $O(1)$

```
// faster than unordered_map, cannot use .count()
#include <ext/pb_ds/assoc_container.hpp>
using namespace __gnu_pbds;
gp_hash_table<int,int> hashmap;
gp_hash_table<int,null_type> hashset;
```

8.2 Ordered Set (pb_ds)

Time Complexity: $O(\log N)$

```
// k번째 원소확인 및 x보다 작은 원소개수확인 O(logN)
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;
template <typename T>
using ordered_set = tree<T, null_type, less<T>, rb_tree_tag,
tree_order_statistics_node_update>;
// ordered_set<int> os;
// os.find_by_order(k): k번째 원소 iter(0-idx, 없으면
os.end())
// os.order_of_key(x) : x  미만 개수
template <typename T>
using ordered_mset = ordered_set<pair<T, int>>;
template <typename T>
auto m_find(ordered_mset<T>& os, T x) {
  auto it = os.lower_bound({x, 0});
  return it != os.end() && it->first == x ? it : os.end();
}
// ordered_mset<int> oms; int id = 0;
// oms.insert({x, id++}); auto it = m_find(oms, x);
// if (it != oms.end()) oms.erase(it);
// oms.find_by_order(k): k번째 값 (0-idx), k < size
// oms.order_of_key({x, 0} or {x, INF}) : x 미만 or 이하 개수
```

8.3 Priority Queue (pb_ds)

Usage: Meldable heap supporting modify/erase via point_iterator.
Time Complexity: $O(\log N)$

```
// 큐 병합, 임의 값 수정 및 삭제 가능
#include <ext/pb_ds/priority_queue.hpp>
using namespace __gnu_pbds;
template <typename T, typename Comp = less<T>>
using pbds_pq = __gnu_pbds::priority_queue<T, Comp,
pairing_heap_tag>;
// pbds_pq<int> mx; / pbds_pq<int, greater<int>> mn;
pbds_pq<int> pq1, pq2;
auto it = pq1.push(10); pq2.push(100);
pq1.join(pq2); // O(1), pq1: {10, 100}, pq2: {}
pq1.modify(it, 50); // O(logN), pq1 : {50, 100}
pq1.erase(it); // O(logN), pq1: {100}
pq1.top(); pq1.empty(); pq1.size(); pq1.pop();
```

8.4 Rope

Time Complexity: $O(\log N)$

```
#include <ext/rope>
using namespace __gnu_cxx;
int main() {
  string str; crope r(str.c_str()); // vector<T> v(N);
  rope<T> r(v.data(), N);
  r.insert(pos, str); r.erase(pos, len); // O(logN)
  r.replace(pos, len, str); // O(logN)
  crope r2 = r; // O(1)
  r2 = r.substr(pos, len); // O(logN)
  r += r2; // Append O(logN)
  r[idx]; // O(logN), but for(auto i : r) is O(N)
  cout << r; // O(N)
}
```

9 Misc

9.1 Custom Hash

Usage: Custom hash for pair, vector.

```
struct custom_hash {
  template <class T>
  void combine(size_t& seed, const T& v) const {
    seed ^= hash<T>{}(v) + 0x9e3779b9 + (seed<<6) + (seed>>2);
  }
  template <class T1, class T2>
  size_t operator()(const pair<T1, T2>& p) const {
    size_t seed = 0; combine(seed, p.first);
    combine(seed, p.second); return seed;
  }
  template <class T>
  size_t operator()(const vector<T>& v) const {
    size_t seed = 0;
    for (const auto& i : v) combine(seed, i);
    return seed;
  }
};
```

9.2 Fast I/O

```
struct FastIO {
  static constexpr int SZ = 1 << 16;
  char rb[SZ], wb[SZ];
  char *p1 = rb, *p2 = rb, *pp = wb;
  inline char getc() {
    return p1 == p2 && (p2 = (p1 = rb) + fread(rb, 1, SZ,
    stdin), p1 == p2) ? -1 : *p1++;
  }
  inline void putc(char c) {
    if (pp == wb + SZ) fwrite(wb, 1, SZ, stdout), pp = wb;
    *pp++ = c;
  }
  void read(auto &x) {
    x = 0; char c = getc(); bool f = 0;
    while (c != -1 && (c < '0' || c > '9')) { if (c == '-')
    f = 1; c = getc(); }
    while (c >= '0' && c <= '9') x = x * 10 + (c ^ 48), c =
    getc();
    if (f) x = -x;
  }
  void write(auto x) {
    if (x < 0) putc('-'), x = -x;
    static char t[20]; int ti = 0;
    do t[ti++] = (x % 10) ^ 48; while (x /= 10);
    while (ti) putc(t[--ti]);
  }
  ~FastIO() { fwrite(wb, 1, pp - wb, stdout); }
} io;
```

9.3 Random

```
#include <random>
#include <chrono>
mt19937_64
rng(chrono::steady_clock::now().time_since_epoch().count());
uniform_int_distribution<int>(l, r)(rng); // [l, r]
uniform_real_distribution<double>(l, r)(rng); // [l, r)
shuffle(all(v), rng); // shuffle vector
vector<double> w = { 40, 10, 50 };
discrete_distribution<int>(all(w))(rng); // 0: 40%, 1: 10%,
2: 50%
```

9.4 Ternary Search

Time Complexity: $O(\log N)$

```
ll ternary_search(ll lo, ll hi, auto f) {
  if (lo > hi) return 0;
  while (hi - lo >= 3) {
    ll p = lo + (hi-lo) / 3, q = hi - (hi-lo) / 3;
    if (f(p) < f(q)) hi = q; // for max: f(p) > f(q)
    else lo = p;
  }
  ll res = lo; // for max: f(i) > f(res)
  for (ll i = lo+1; i <= hi; i++) if (f(i) < f(res)) res = i;
  return res;
}
double ternary_search(double lo, double hi, auto f, int
it=100) {
  while (it--) {
    double p = (lo*2 + hi) / 3., q = (lo + hi*2) / 3.;
    if (f(p) < f(q)) hi = q; // for max: f(p) > f(q)
    else lo = p;
  }
  return (lo+hi) / 2.;
}
```

9.5 Some tricks

```
// ll에는 ~ll 사용, clz/ctz는 x=0에서 사용 불가.
__builtin_popcount(x); // 켜진 비트(1)의 총 개수
__builtin_clz(x); // 왼쪽(MSB)부터 연속된 0의 개수
__builtin_ctz(x); // 오른쪽(LSB)부터 연속된 0의 개수
// popcount를 유지하면서 다음으로 큰 수 / 작은 수
// min: (1LL << k)-1, max: ((1LL << k)-1) << (n-k);
bool next_comb(ll& bit, int N) {
  ll x = bit & -bit, y = bit + x;
  bit = (((bit & ~y) / x) >> 1) | y;
  return (bit < (1LL << N));
}
bool prev_comb(ll& bit) {
  ll y = ~bit & -~bit, x = bit & -y;
  bit = x - ((x & -x) / (y << 1));
  return x != 0;
}
// mask의 모든 부분집합을 내림차순으로 순회 (0 제외), O(3^N)
for (int sub = mask; sub > 0; sub = (sub-1)&mask);
// mask를 포함하는 모든 상위집합을 오름차순으로 순회
for (int sup = mask; sup < (1<<n); sup = (sup+1)|mask);
// 런타임 변수 n에 맞는 크기의 bitset을 사용
template <int len = 1> void solve(int n) {
  if (len < n) { solve<min(len*2, 200005)>(n); return; }
  bitset<len> bs;
  // do stuff
}
// bitset 고속순회 (켜져있는 비트만 순회)
for (int i = bs._Find_first(); i < bs.size(); i =
bs._Find_next(i)) {
  // do stuff
}
// 1부터 n까지의 수에서 숫자 i가 등장하는 총 횟수
ll count_digit_frq(ll n, int i) {
  ll ret = 0;
  for (ll j = 1; j <= n; j *= 10) {
    ll q = n / (j*10), r = n % (j*10);
    ret += (i == 0 ? (q-1)*j : q*j);
    if (r >= i*j) ret += (r < (i+1)*j ? r-i*j+1 : j);
  }
  return ret;
}
// 특정 날짜(년, 월, 일)의 요일 / 0: Sat, 1: Sun, ...
int get_day_of_week(int y, int m, int d) {
  if (m <= 2) y--, m += 12; int c = y / 100; y %= 100;
  int w = ((c>>2)-(c<<1)+y+(y>>2)+(13*(m+1)/5)+d-1)%7;
  if (w < 0) w += 7; return w;
}
// 도달 가능 여부 O(N^3 / 64)
bitset<MAXN> reach[MAXN];
for (int k = 0; k < n; k++) {
  for (int i = 0; i < n; i++) {
    if (reach[i][k]) reach[i] |= reach[k];
  }
}
sort(all(v)); // Permutation
do {
  // process v
} while (next_permutation(all(v)));
vector<int> mask(n, 0); // Combination(nCk)
fill(mask.end()-r, mask.end(), 1); // pick r
do {
  for (int i = 0; i < n; i++) if (mask[i]) {
    /* v[i] is selected */ }
} while (next_permutation(all(mask)));
sort(all(v)); // Partial Permutation (nPk)
do {
  for(int i = 0; i < k; i++) { /* use v[i] */ }
  reverse(v.begin()+k, v.end());
} while (next_permutation(all(v)));
// 부호 있는 정수 나눗셈의 floor/ceil. b!=0, 몫은 ll 범위.
```

```cpp
ll floor_div(ll a, ll b) {
  ll q = a/b, r = a%b;
  return q - (r && (r<0) != (b<0));
}
ll ceil_div(ll a, ll b) {
  ll q = a/b, r = a%b;
  return q + (r && (r<0) == (b<0));
}
// 부분배열 GCD/AND/OR의 결과별 개수·합 집계.
// v: 현재 위치에서 끝나는 부분배열의 {결과 값, 개수}, 짧은 구간부터.
vector<pair<ll,ll>> v;
ll ans = 0;
for (ll x : a) {
  vector<pair<ll,ll>> nv{{ x, 1 }};
  for (auto [g, c] : v) {
    g = gcd(g, x); // AND: g &= x; OR: g |= x;
    if (nv.back().first == g) nv.back().second += c;
    else nv.push_back({ g, c });
  }
  v.swap(nv);
  for (auto [g, c] : v) ans += g*c; // 모든 부분배열 GCD의 합
}
```

10 Checklist + Useful Info

10.1 Highly Composite Numbers, Large Prime

< 10^k	number	divs	2 3 5 7 11 13 17 19 23 29 31 37 41 43 47 53 59 61 67 71
1	6	4	1 1
2	60	12	2 1 1
3	840	32	3 1 1 1
4	7560	64	3 3 1 1
5	83160	128	3 3 1 1 1
6	720720	240	4 2 1 1 1 1
7	8648640	448	6 3 1 1 1 1
8	73513440	768	5 3 1 1 1 1 1
9	735134400	1344	6 3 2 1 1 1 1
10	6983776800	2304	5 3 2 1 1 1 1 1
11	97772875200	4032	6 3 2 2 1 1 1 1
12	963761198400	6720	6 4 2 1 1 1 1 1 1
13	9316358251200	10752	6 3 2 1 1 1 1 1 1 1
14	97821761637600	17280	5 4 2 2 1 1 1 1 1 1
15	866421317361600	26880	6 4 2 1 1 1 1 1 1 1 1
16	8086598962041600	41472	8 3 2 2 1 1 1 1 1 1 1
17	74801040398884800	64512	6 3 2 2 1 1 1 1 1 1 1 1
18	897612484786617600	103680	8 4 2 2 1 1 1 1 1 1 1 1

< 10^k	prime	# of prime	< 10^k	prime
1	7	4	10	9999999967
2	97	25	11	99999999977
3	997	168	12	999999999989
4	9973	1229	13	9999999999971
5	99991	9592	14	99999999999973
6	999983	78498	15	999999999999989
7	9999991	664579	16	9999999999999937
8	99999989	5761455	17	99999999999999997
9	999999937	50847534	18	999999999999999989

10.2 Useful Stuff

- Catalan Number
  1, 1, 2, 5, 14, 42, 132, 429, 1430, 4862, 16796, 58786, 208012, 742900
  $C_n = \binom{2n}{n}/(n+1)$;
  $C_n = \sum_{i=0}^{n-1} C_i \times C_{n-1-i}$;
  - 길이가 2n인 올바른 괄호 수식의 수
  - n + 1개의 리프를 가진 풀 바이너리 트리의 수
  - n + 2각형을 n개의 삼각형으로 나누는 방법의 수
  - n*n 격자에서 0,0부터 n,n까지 이동할 때, y=x(대각선)를 넘지않고 우상단으로만 이동하는 경로의 수
  - n개의 노드로 이루어진 이진 트리 구조 개수
  - 1부터 n까지의 수를 스택에 넣고 빼서 만들 수 있는 순열의 개수
  - 원 위에 2n개의 점이 있고, 2개씩 짝지어 선분을 그었을때 어떤 선분도 교차하지 않는 방법의 수
  - $(0,0)$에서 (N,M)까지 우상단으로 이동할 때, 직선 $y = x + k (k > 0)$에 닿지않는 경로의 수(M N+k이면 0, 그 외의 경우): 전체 경로 수 - 반사된 경로 수 = $\binom{N+M}{N} - \binom{N+M}{N+k}$ (직선 $y = x + k$에 처음 닿은 이후 경로를 $y = x + k$에 대해 대칭 이동 -> $(M-k, N+k)$로 매핑됨)
- Derangements
  N명의 사람이 모자를 벗어두었다가 무작위로 하나씩 집었을 때, 아무도 자기 모자를 쓰지 않을 경우의 수
  - 포함배제 ($D_N = N! \sum_{i=0}^{N} \frac{(-1)^i}{i!}$) - 팩토리얼 역원 전처리로 O(N)
  - DP ($D_N = (N-1) \times (D_{N-1} + D_{N-2})$ (단, $D_0 = 1, D_1 = 0$)) - O(N) dp 테이블을 만들어두면 쿼리 당 O(1)
  - 부분 교란 순열: N명 중 정확히 K명만 자기 모자를 쓰고, 나머지 $N-K$명은 아무도 자기 모자를 쓰지 않을 경우의 수 - $\binom{N}{K} \times D_{N-K}$
- Stars and Bars
  N개의 구별되지 않는 물건을 K개의 구별되는 상자에 나누어담는 경우의 수 (빈 상자 허용) - $\binom{N+K-1}{K-1}$
  - $x_1 + x_2 + \cdots + x_K = N$ ($x_i \geq 0$)의 해의 개수를 구하는 것과 동일
  - 각 상자에 적어도 1개 이상 담아야 한다($x_i \geq 1$): 미리 1개씩 나눠줌. $N' = N - K$로 치환해 $\binom{N'+K-1}{K-1} = \binom{N-1}{K-1}$ 계산
  - 어떤 상자도 C개를 초과할 수 없다($0 \leq x_i \leq C$): 포함배제로 계산.

$$\sum_{j=0}^{\min(K, \lfloor N/(C+1) \rfloor)} (-1)^j \binom{K}{j} \binom{N - j(C+1) + K - 1}{K - 1}$$

  (단, $N, C \geq 0, K \geq 1$)
- 제2종 스털링 수
  서로 다른 N개의 물건을 구별되지 않는 K개의 상자에 빈 상자 없이 나누어 담는 경우의 수 ($S(N,K)$ 혹은 $\left\{ {N \atop K} \right\}$)
  - 점화식: $S(N,K) = K \times S(N-1,K) + S(N-1,K-1)$ ($S(0,0) = 1, S(N,0) = 0$ ($N > 0$))
  - 직접 계산: $S(N,K) = \frac{1}{K!} \sum_{i=0}^{K} (-1)^{K-i} \binom{K}{i} i^N$ (특정 $S(N,K)$ 하나 계산 시 $O(K \log N)$)
  - Bell Number ($B_N = \sum_{k=0}^{N} S(N,k)$): 서로 다른 N개의 원소를 분할하는 모든 방법의 수.
  - Bell Number 초항: 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975
- 자연수의 분할
  서로 같은 N개의 물건을 구별되지 않는 K개의 상자에 빈 상자 없이 나누어 담는 경우의 수 ($P(N,K)$)
  - 자연수 N을 K개의 자연수의 합으로 나타내는 방법의 수와 동일.
  - 점화식: $P(N,K) = P(N-1,K-1) + P(N-K,K)$
  - $P(0,0) = 1, P(N) = \sum_{K=0}^{N} P(N,K)$: 자연수 N의 총 분할 수.
  - $P(N)$ 초항: 1, 1, 2, 3, 5, 7, 11, 15, 22, 30, 42, 56, 77, 101, 135
- Cayley's Formula & Prüfer Sequence
  - N개의 구별되는 정점으로 만들 수 있는 서로 다른 스패닝 트리의 개수 $= N^{N-2}$
  - 각 정점의 차수가 $d_1, d_2, \ldots, d_N$으로 고정되었을 때 가능한 트리의 개수 $= \frac{(N-2)!}{(d_1-1)!(d_2-1)!\ldots(d_N-1)!}$ (단, $\sum d_i = 2N-2$)
- Binomial Identities
  시그마가 포함된 조합론 수식을 $O(1)$로 단순화할 때 사용.
  - Hockey-stick Identity (파스칼의 삼각형 대각선 합): $\sum_{i=K}^{N} \binom{i}{K} = \binom{N+1}{K+1}$
  - Vandermonde's Identity (두 집합에서 뽑기): $\sum_{i=0}^{K} \binom{N}{i} \binom{M}{K-i} = \binom{N+M}{K}$
  - 제곱의 합: $\sum_{i=0}^{N} \binom{N}{i}^2 = \binom{2N}{N}$
- Fibonacci Identities
  - Cassini's identity: $F_{n-1}F_{n+1} - F_n^2 = (-1)^n$
  - Addition formula: $F_{n+k} = F_k F_{n+1} + F_{k-1} F_n$
  - Zeckendorf's theorem: 모든 양의 정수는 연속하지 않는 피보나치 수들(1,2,3,5,...)의 합으로 유일하게 표현 가능하며, 이는 그리디하게 큰 피보나치 수부터 빼는 것으로 구할 수 있다.
- Möbius Inversion
  - $g(n) = \sum_{d \mid n} f(d) \iff f(n) = \sum_{d \mid n} \mu(d) g(n/d)$
  - $g(n) = \sum_{n \mid d} f(d) \iff f(n) = \sum_{n \mid d} \mu(d/n) g(d)$
- Burnside' s Lemma
  경우의 수를 세는데, 특정 transform operation(회전, 반사, ..) 해서 같은 경우들은 하나로 친다. 전체 경우의 수는? 각 operation마다 이 operation을 했을 때 변하지 않는 경우의 수를 센다 (단, "아무것도 하지 않는다" 라는 operation도 있어야 함!) 전체 경우의 수를 더한 후, operation의 수로 나눈다. (답이 맞다면 항상 나누어 떨어져야 한다)
- 알고리즘 게임
  - Nim Game의 해법: 각 더미의 돌의 개수를 모두 XOR했을 때 0이 아니면 첫번째, 0이면 두번째 플레이어가 승리.
  - Grundy Number: 어떤 상황의 Grundy Number는, 가능한 다음 상황들의 Grundy Number를 모두 모은 다음, 그 집합에 포함되지 않는 가장 작은 수가 현재 state의 Grundy Number가 된다. 만약 다음 state가 독립된 여러개의 state들로 나뉠 경우, 각각의 state의 Grundy Number의 XOR 합을 생각한다.
  - Subtraction Game: 한 번에 k 개까지의 돌만 가져갈 수 있는 경우, 각 더미의 돌의 개수를 k + 1로 나눈 나머지를 XOR 합하여 판단한다.
  - Index-k Nim: 한 번에 최대 k개의 더미를 골라 각각의 더미에서 아무렇게나 돌을 제거할 수 있을 때, 각 binary digit에 대하여 합을 k + 1로 나눈 나머지를 계산한다. 만약 이 나머지가 모든 digit에 대하여 0이라면 두번째, 하나라도 0이 아니라면 첫번째 플레이어가 승리. - Misere Nim: 모든 돌 무더기가 1이면 N이 홀수일 때 후공 승, 그렇지 않은 경우 XOR 합 0이면 후공 승
- Pick' s Theorem
  격자점으로 구성된 simple polygon이 주어짐. I 는 polygon 내부의 격자점 수, B 는 polygon 선분 위 격자점 수, A는 polygon의 넓이라고 할 때, 다음과 같은 식이 성립한다. 경계 격자점 수 $B = \sum_{edges} gcd(|dx|, |dy|)$. 내부 격자점 수 $I = A - B/2 + 1$.

- 홀의 결혼 정리: 이분그래프(L-R)에서, 모든 L을 매칭하는 필요충분 조건 = L에서 임의의 부분집합 S를 골랐을 때, 반드시 (S의 크기) ≤ (S와 연결되어있는 모든 R의 크기)이다.
- 소수: 10 007 , 10 009 , 10 111 , 31 567 , 70 001 , 1 000 003 , 1 000 033 , 4 000 037 , 99 999 989 , 999 999 937 , 1 000 000 007 , 1 000 000 009 , 9 999 999 967 , 99 999 999 977
- 소수 개수 : (1e5 이하 : 9592), (1e7 이하 : 664 579) , (1e9 이하 : 50 847 534)
- N의 약수의 개수 = $O(N^{1/3})$, N의 약수의 합 = $O(N \log\log N)$
- $\gcd(m,n)=1$ 이면 $\phi(mn)=\phi(m)\phi(n)$
  소수 p, 정수 $r \geq 1$에 대해 $\phi(p^r)=p^r-p^{r-1}$
  $\gcd(a,n)=1$ 이면 $a^{\phi(n)} \equiv 1 \pmod n$
- Euler's phi $\phi(n)=n\prod_{p|n}\left(1-\frac{1}{p}\right)$
- Euler characteristic : v - e + f (면, 외부 포함) = 1 + c (컴포넌트)
- Lucas' Theorem $\binom{m}{n} = \prod \binom{m_i}{n_i} \pmod p$ m_i, n_i는 p^i의 계수
- 스케줄링에서 데드라인이 빠른 걸 쓰는게 이득. 늦은 스케줄이 안들어갈 때 가장 시간 소모가 큰 스케줄 1개를 제거하면 이득.
- 2차원 누적합 / imos ($i=$ 행, $j=$ 열, 1-based)
  누적합 생성: $S[i][j]=S[i-1][j]+S[i][j-1]-S[i-1][j-1]+A[i][j]$
  구간합 $[i_1,j_1] \sim [i_2,j_2]$: $S[i_2][j_2]-S[i_1-1][j_2]-S[i_2][j_1-1]+S[i_1-1][j_1-1]$
  imos ($+v$), 부호 패턴 $+,-,-,+$:
  $d[i_1][j_1] \mathrel{+}= v,\ d[i_2+1][j_1] \mathrel{-}= v,\ d[i_1][j_2+1] \mathrel{-}= v,\ d[i_2+1][j_2+1] \mathrel{+}= v$
  (imos는 마지막에 누적합 2번 적용: 행 방향, 열 방향)
- 순열 swap 관련
  - 버블소트 최소 swap = inversion 개수
  - 순열 정렬 최소 swap = $N-$(사이클 개수)
- 나머지 그래프
  무제한 동전 조합에서 목표 금액 M이 너무 커 금액별 DP가 불가능할 때, 동전 L을 더해도 나머지가 같다는 점으로 상태를 L개로 압축.
  정점 $r=$ 금액 mod L, 동전 a 추가는 $r \to (r+a) \bmod L$. 0에서 다익스트라.
  - 표현 판정: L은 아무 동전(보통 최소), 비용 a. $dist[r]$는 최소 금액이므로 $dist[M \bmod L] \leq M$이면 가능.
  - 최소 개수: L은 최대 동전, 비용 $L-a$(최대 동전 대비 부족분). $M \geq (L-1)^2$이면 답은 $(M+dist[M \bmod L])/L$. 작은 M은 DP, 도달 불가면 불가능.
- Bounded Scheduling
  현재 자원 $x \geq B_i$이면 작업 i 수행 가능, 수행 후 $x \leftarrow x+A_i$. - $A_i \geq 0$인 작업을 먼저 B_i 오름차순으로 처리.
  - $A_i < 0$인 작업은 A_i+B_i 내림차순으로 처리.
  이 순서에서 수행 불가능한 작업이 나오면 다른 순서로도 불가능.
- 이항계수 구간합
  $F(n,l,r)=\sum_{k=l}^{r}\binom{n}{k}$. 여러 쿼리를 Mo처럼 처리할 때 n,l,r을 한 칸씩 이동하며 갱신:
  - $F(n,l,r+1)=F(n,l,r)+\binom{n}{r+1}$.
  - $F(n,l+1,r)=F(n,l,r)-\binom{n}{l}$.
  - $F(n+1,l,r)=2F(n,l,r)+\binom{n}{l-1}-\binom{n}{r}$.
  - $F(n-1,l,r)=\left(F(n,l,r)-\binom{n-1}{l-1}+\binom{n-1}{r}\right)/2$.

  범위 밖의 이항계수는 0. 모듈러 계산의 /2는 2의 역원 사용.

10.3 자주 쓰이는 문제 접근법

- 작은 예제와 brute force부터: 직접 푸는 과정을 수식화하고, 비슷한 문제를 떠올리기.
- 특수 조건, 여사건, 역순, 순서 강제, 답의 정규화로 문제를 단순화하기.
- DP 상태/전이, 그리디, 그래프/플로우, 자료구조로 모델링하고 시간복잡도 확인하기.
- 막히면 접근을 바꾸거나 헬프콜. 당연하게 둔 가정과 빠진 경우를 다시 확인하기.
- 구간을 통째로 가져간다 : 플로우+@ $(i, i+1, k, 0)$, $(s, e, 1, w)$, $(N, T, k, 0)$.

10.4 DP 최적화 접근

- C[i, j] = A[i] * B[j]이고 A, B가 단조증가, 단조감소이면 Monge
- l..r의 값들의 sum이나 min은 Monge
- 식 정리해서 일차(CHT) 혹은 비슷한(MQ) 함수를 발견, 구현 힘들면 Li-Chao
- $a \leq b \leq c \leq d$에서 $A[a,c]+A[b,d] \leq A[a,d]+A[b,c]$
- Monge 성질을 보이기 어려우면 N^2 나이브 짜서 opt의 단조성을 확인
- 식이 간단하거나 변수가 독립적이면 DP 테이블을 세그 위에 올려서 해결
- 침착하게 점화식부터 세우고 Monge인지 판별
- Monge에 집착하지 말고 단조성이나 볼록성만 보여도 됨

10.5 Graph Matching(Graph with $|V| \leq 500$)

- **Game on a Graph**: 정점 s에서 시작해, 번갈아 토큰을 방문하지 않은 인접 정점으로 이동. 이동하지 못하면 패배.
  s를 제외해도 최대 매칭 크기가 같으면 후공 승, 아니면 선공 승. `match()` 이후 `chk(s)`로 판정.
- **Minimum Edge Cover**: 모든 정점이 선택한 간선 하나 이상과 인접하도록 하는 최소 간선 집합.
  최대 매칭 M을 구하면 답은 $|V|-|M|$. 복원: 매칭 간선을 선택하고, 매칭되지 않은 정점마다 인접 간선 하나 추가. 고립 정점이 있으면 불가능.
- **기계당 노동자 2명 배정 (NEERC'18 B)**: 노동자마다 담당 가능한 기계가 주어지고, 각 노동자는 최대 한 기계에 배정. 노동자 2명을 배정한 기계 수 최대화.
  노동자마다 정점 하나, 기계마다 정점 두 개를 만들고 기계 내부 두 정점을 연결. 각 노동자를 담당 가능한 기계의 두 정점 모두와 연결.
  최대 매칭 M을 구하면 답은 $|M|-$ 기계 수.

10.6 Mincut 모델링

- N개의 boolean 변수 $v_1, \cdots, v_n$을 정해서 비용을 최소화하는 문제=true인 점은 T, false인 점은 F와 연결되게 분할하는 민컷 문제
  1. v_i가 T일 때 비용 발생: i에서 F로 가는 비용 간선
  2. v_i가 F일 때 비용 발생: T에서 i로 가는 비용 간선
  3. v_i가 T이고 v_j가 F일 때 비용 발생: i에서 j로 가는 비용 간선
  4. $v_i \neq v_j$ 일 때 비용 발생: i에서 j로, j에서 i로 가는 비용 간선
  5. v_i가 T면 v_j도 T여야 함($v_i \implies v_j$): i에서 j로 가는 무한 간선
  6. v_i가 F면 v_j도 F여야 함($v_j \implies v_i$): j에서 i로 가는 무한 간선
  7. v_i가 무조건 T/F: T에서 i / i에서 F로 가는 무한 간선
  8. $v_i = v_j$ 여야 함: $i \to j, j \to i$ 양방향 무한 간선
  9. $v_i \neq v_j$ 여야 함: 이분 그래프만 가능. $v_j' = \neg v_j$, $v_i = v_j'$로 치환 ($i \leftrightarrow j$ 무한 간선)
  10. $\neg(v_i \wedge v_j)$ (둘 다 T 금지): 이분 그래프만 가능. $v_j' = \neg v_j, v_i \implies v_j'$ ($i \to j$ 무한 간선)
  11. $v_i \vee v_j$ (둘 다 F 금지): 이분 그래프만 가능. $v_j' = \neg v_j, v_j' \implies v_i$ ($j \to i$ 무한 간선)
- 두 변수 비용표: E_{ab}는 $i=a, j=b$일 때 비용. T쪽은 1, F쪽은 0.
  $E_{00}+E_{11} \leq E_{01}+E_{10}$이면 다음 간선 추가:
  - $i \to j$: 용량 $E_{01}+E_{10}-E_{00}-E_{11}$.
  - $i \to F$: $\max(0, E_{11}-E_{01})$, $T \to i$: $\max(0, E_{01}-E_{11})$.
  - $j \to F$: $\max(0, E_{01}-E_{00})$, $T \to j$: $\max(0, E_{00}-E_{01})$.

  원래 최소 비용 = 최소컷 $+E_{00}+\min(0, E_{11}-E_{01})+\min(0, E_{01}-E_{00})$.
- 모든 조건을 만족하는지 판단하면 2-SAT. 만족하는 절의 개수 · 가중치를 최대화하면 MAX-2SAT
- Maximum Density Subgraph (NEERC'06H, BOJ 3611 팀의 난이도)
  – N=정점 수, M=간선 수. M/N을 최대화. 밀도 d를 이분탐색.
  정점 N개에 S,T를 추가해 다음 용량으로 연결:
  - S → v : M, v → T : M-deg[v]+2d
  - 원본 간선 u-v : u→v, v→u 각각 1

  maxflow < M*N이면 밀도>d인 해가 존재 → d를 높임.
  성공할 때마다 잔여 그래프에서 S가 도달하는 원본 정점들을 저장.
  M=0이면 정점 하나가 답.